Text-to-SQL Agent — Project Document & 10-Day Claude Code Build Plan

Oct 6, 2026 · @Sarang Chothe

How to read this document. Sections 1 to 11 define what is built, section 12 explains how three Claude Code sessions build it in parallel, section 13 is the day-by-day plan for all three tracks, and sections 14 to 16 hold the templates, risks and appendix. Everything in this document is built within the 10 days; nothing is deferred.

1. Executive summary

This plan builds a complete, local-only text-to-SQL platform for PostgreSQL in 10 working days: a FastAPI service running a LangGraph pipeline over a semantic layer, a semantic compiler that is the only route to the database, login with local accounts, LDAP and single sign-on, custom roles and row-level security, scheduled profiling with drift detection, dashboards with scheduled questions and sharing, a visual modeling canvas, an audit browser and an evaluation harness. Everything in this document is built inside the 10 days; no feature is deferred to a later phase.

Feasibility. This is a large scope for ten days, and what makes it possible is stated openly. Three tracks run in parallel, each driven by its own Claude Code session in a separate git worktree: Track A (semantic engine and agent), Track B (platform: identity, roles, policies, scheduling, sharing) and Track C (web UI). Every feature has a deliberately narrow definition of done (section 13). Each day ends with a merge and review gate. The limiting resource is review time, not code generation: one person reviewing three tracks has little slack, so the afternoon of Day 9 and Day 10 are held as buffer, and a second reviewer for Track B is strongly recommended. A ticket that slips carries into the next day on the same track; nothing is removed from the plan.

Local-only runtime. Nothing at runtime calls a cloud service: the LLM, the embedding model, Qdrant, the metadata database, the identity provider, tracing and the UI all run on your machine or network, and a test proves it by blocking outbound traffic. Claude Code is a development tool only; if your build tooling must also be local, replace it with a local coding assistant and rework section 12. Local models are weaker and slower than hosted ones, so the plan adds a model bake-off on Day 1, relies on the semantic layer and retrieval to keep prompts short, and sets accuracy and latency targets from measured baselines instead of assumptions.

Why this shape. Accuracy in text-to-SQL depends more on three design choices than on a bigger prompt: the LLM sees only a curated business vocabulary (the semantic layer), every generated query is checked against that vocabulary before it touches the database,

and failures are fed back as structured errors for correction. The same design makes governance enforceable: because the model writes SQL over logical models and a compiler turns it into the real query, row-level security predicates are injected by the compiler and cannot be bypassed by anything the model writes.

Key decisions

Decision	Choice	Reason
Orchestration	LangGraph `StateGraph` with a checkpointer	Explicit retry loop, resumable threads, node-level streaming
Architecture	Hexagonal (ports and adapters)	Swap the LLM, vector store or identity provider; mock everything in tests
Warehouse	PostgreSQL only: one `WarehousePort` with a psycopg adapter; any number of Postgres databases as projects	One dialect to get right; no other connectors are built
Semantic layer	JSON MDL validated by Pydantic, edited visually on a modeling canvas	One source of truth for the LLM, the compiler and the validator
SQL path	The LLM writes SQL over logical models; a semantic compiler (sqlglot) expands relationships, calculated fields and metrics, injects row-level security predicates and emits PostgreSQL SQL	Removes naming and join errors and makes row policies impossible to bypass
Vector store	Qdrant, payload-filtered by project and model version	Tenant scoping by filter
LLMs	Local open-weight models served by Ollama: a coder-class model for SQL, a smaller instruct model for intent and rewrite; names in config	No data leaves the machine; model size matched to task difficulty
Identity	Local accounts, LDAP bind and OIDC single sign-on through a self-hosted provider	Works offline and fits existing directories

Decision	Choice	Reason
Authorization	Built-in roles DBA and user, custom roles built from capabilities, row-level security	Users only ask, DBAs configure, custom roles cover the cases in between
Background work	One worker process (APScheduler, jobs stored in the metadata database)	Scheduled profiling and scheduled questions without extra infrastructure
Streaming	Server-Sent Events read with fetch streaming	Node-level progress while still sending the Authorization header
Dev method	Three Claude Code sessions in git worktrees, a daily merge gate, `CLAUDE.md`, subagents, hooks (section 12)	Parallel work without collisions; tests and review keep it consistent

Targets at Day 10 (assumptions to confirm)

- Execution accuracy on a 40-question golden set, measured through the compiler path: the target is fixed on Day 3 from the chosen model's measured baseline (provisional goal 70%, stretch 80%).
- 0 executed non-SELECT statements across the red-team set, and 0 rows outside a row policy across the row-level security suite.
- First streamed event under 2 s; p50 end-to-end answer under 20 s on the reference hardware (section 4).
- Every role, login method and scheduled job demonstrated end to end in Playwright and in the offline test.
- One docker compose up brings up the API, worker, UI, Qdrant, metadata database, local model server and demo database, and keeps working with no internet access once the models are pulled.

Deliverables at Day 10

- Repository with CI, `make check` and `make eval`, and a tagged `v0.1.0` release.
- The compose stack, with an identity-demo profile that adds OpenLDAP and Keycloak for testing LDAP and single sign-on.
- OpenAPI specification and a generated TypeScript client.
- Committed evaluation report covering the golden set, intent set, follow-up chains, red-team set and row-level security suite.
- README, runbook and a 10-minute demo script; this document, updated to match what was built.

2. Scope

At Day 10 a DBA can connect PostgreSQL databases, curate a semantic model on a visual canvas, set governance rules and schedule work, and a user can log in and ask questions in a chat thread that returns validated SQL, a result table, a short answer and optionally a chart. Dashboards, scheduled questions and sharing build on the same pipeline. Everything in this section is built inside the 10 days.

Goals

- Natural-language question to correct, safe, executed SQL over a curated semantic model.
- Transparent steps: the user sees intent, retrieved context, SQL, compilation, validation and corrections as they stream.
- Teachable: a thumbs-up proposes a SQL pair that a DBA approves; business rules are stored as instructions.
- Governed: roles, custom roles, row-level security and a complete audit trail.
- Operable: scheduled profiling with drift detection, scheduled questions, dashboards and sharing.
- Measurable: a repeatable evaluation run produces accuracy, latency and cost numbers.

Boundaries (decisions, not deferrals; none of these is a feature that was left out of the plan)

- PostgreSQL is the only warehouse connector. Because no other connector exists, the compiler emits PostgreSQL SQL only; the target dialect is a parameter in the code, set to PostgreSQL and tested only for PostgreSQL.
- All warehouse access is read-only; the product never writes to a connected database.
- No cloud-hosted model, API or identity provider is used at runtime.
- Sharing stays inside the installation, to named users or roles; there are no public links.
- Model fine-tuning and custom rerankers are research directions, not product features; section 11 lists the accuracy levers that are tried instead.

Personas

Persona	Role	Need	Main screens
Business analyst	user	Ask questions without writing SQL, trust the answer, open shared dashboards	Chat, shared dashboards

Persona	Role	Need	Main screens
Data modeler	DBA	Describe tables, relationships and metrics so answers are accurate	Connect and Model, Knowledge
Platform engineer	DBA	Deploy, schedule, monitor and evaluate the service	Connect and Model, Scheduled questions, Dashboards, Drift
Security administrator	DBA or a custom role	Control who can do what and see which rows; review activity	Users and Roles, Audit

Roles. The system has two built-in roles, and every account holds exactly one role. A **DBA** adds databases, edits and deploys the semantic model, manages SQL pairs and instructions, defines row policies, runs evaluations, builds dashboards, schedules questions, shares them, manages accounts and roles and reads the audit log. A **user** can only log in, ask questions and open what has been shared with them. A DBA can also create **custom roles** by choosing capabilities from the list in section 8, for example an analyst role that may also build dashboards. LDAP groups and OIDC claims map to roles. There is no separate Admin role: the DBA also manages accounts, roles, the audit log and evaluations, and a custom role that holds only the administration capabilities gives separation of duties when you want it. The permission matrix is in section 8.

User stories

- As any user, I log in with my username and password, my directory account or single sign-on, and stay signed in for the working day.
- As a user, I ask a question and see streamed progress, then SQL, rows, a plain-language answer and a chart, and I ask follow-ups such as "what about last year?".
- As a user, I see only the rows my policy allows, in every answer, dashboard and scheduled result.
- As a user, I open dashboards and saved questions shared with me, read-only.
- As a DBA, I add a PostgreSQL database through a three-step wizard (connect, select tables, define relationships) and get a draft semantic model, or I start from a sample dataset.
- As a DBA, I arrange models and draw relationships on a canvas, define calculated fields and metrics, and deploy a versioned model.
- As a DBA, I save good answers as SQL pairs, add instructions, and approve or reject the pairs users propose.

- As a DBA, I edit the generated SQL and re-run it; a user sees the SQL but cannot change it.
- As a DBA, I schedule profiling and see a drift report when the live schema no longer matches the deployed model.
- As a DBA, I pin answers to a dashboard, schedule a question to run hourly, daily or weekly, and share both with users or roles.
- As a DBA, I create accounts and custom roles, map directory groups to roles, set user attributes and row policies, and review the audit log with filters.

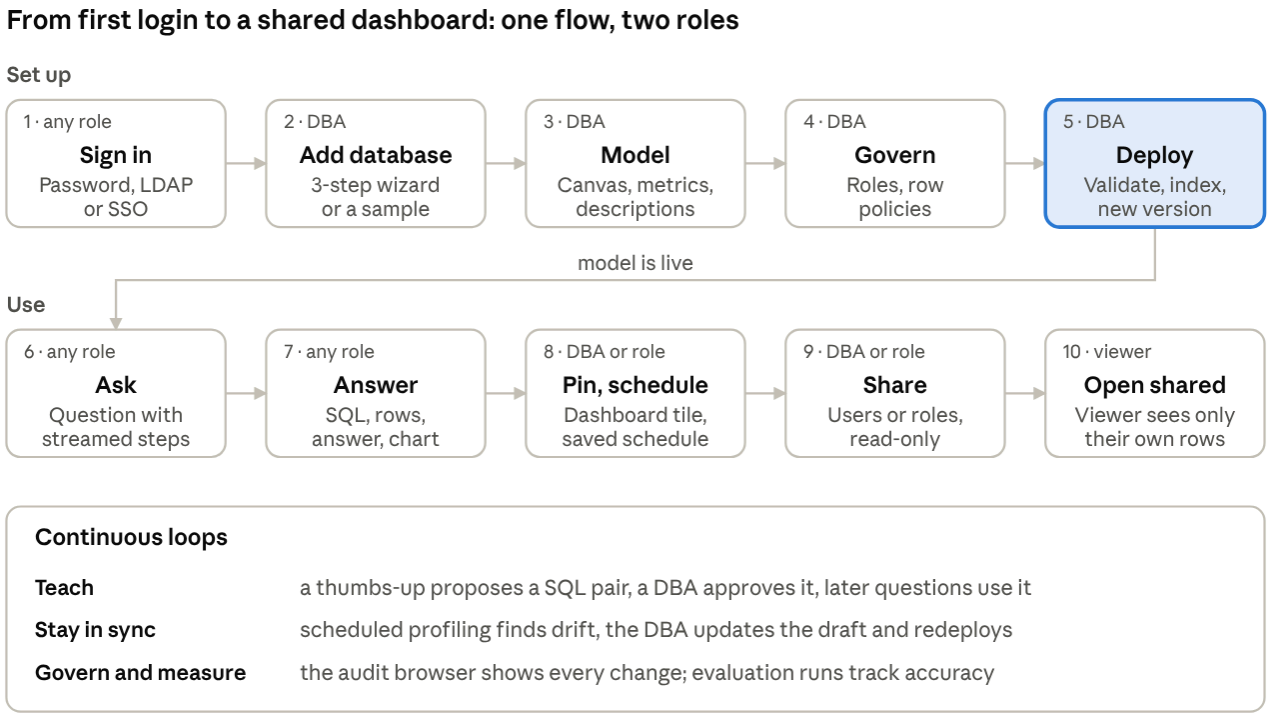
Features built in the 10 days

Area	What is built
Semantic layer	JSON MDL, PostgreSQL introspection, LLM-drafted descriptions, relationships, calculated fields, metrics, a visual modeling canvas, a metrics UI, versioned deploys
Question answering	Intent routing, follow-ups, retrieval, SQL over logical models, semantic compiler, validation and correction, read-only execution, answer, chart, streamed steps
Knowledge	SQL pairs, instructions, proposed pairs approved by a DBA
Warehouse	PostgreSQL only, added through a connect, select tables, define relationships wizard or from a sample dataset; any number of Postgres databases as projects; scheduled profiling with drift reports
Identity and access	Local login, LDAP, OIDC single sign-on, DBA and user roles, custom roles, project access
Data governance	Row-level security policies driven by user attributes and enforced by the compiler; audit events and a full audit browser
Collaboration	Dashboards, scheduled questions, sharing with users or roles (read-only)
Quality	Golden set, intent set, follow-up chains, red-team set, row-level security suite, model bake-off, evaluation reports

Area	What is built
Operations	One-command local stack, offline test, tracing, metrics, runbook

End-to-end product flow

The product is one flow with two halves: a DBA sets up and deploys a model, then everyone asks questions against the live version, and the DBA shares what is worth keeping.



end-to-end flow · 10 steps, 3 loops

The hinge is step 5: nothing is askable until a model version is deployed, and every later answer is computed against that version, under the asker's own role and row policies.

Step	Who	Screen	API calls	Result
1. Sign in	Any account	Login	`POST /auth/login`, or `GET /auth/oidc/start`	A session with the account's role and capabilities
2. Add database	DBA	Setup wizard	`POST /connections/test`, `POST /projects`, `GET /projects/{id}/catalog/tables`, `PUT /projects/{id}/selected-tables`, `POST /projects/{id}/introspect`, `GET /projects/{id}/relationship-suggestions`, `PUT /projects/{id}/relationships`; or `POST /samples/{id}/create`	A project with a draft MDL built from the chosen tables and reviewed relationships

Step	Who	Screen	API calls	Result
3. Model	DBA	Canvas and Metrics	`PUT /projects/{id}/mdl`, `POST /projects/{id}/compile-preview`	Described models, relationships, calculated fields and metrics, each checked by the compiler
4. Govern	DBA	Users and Roles, Row policies	`POST /users`, `/roles`, `PUT /identity/mappings`, `/projects/{id}/row-policies`, `PUT /users/{id}/attributes`	Accounts, custom roles, directory mappings and row policies
5. Deploy	DBA	Connect and Model	`POST /projects/{id}/deploy`	A new active model version, indexed and enforced by the compiler
6. Ask	Any account with `ask_question`	Home	`POST /threads/{id}/ask` (SSE)	Streamed intent, retrieval, SQL, compilation and validation steps
7. Answer	Same	Home	The same stream: `rows`, `answer_token`, `chart`, `done`	Rows limited by the asker's policies, a plain-language answer and a chart
8. Pin and schedule	DBA, or a role with `create_dashboard` and `schedule_question`	Home, Dashboards, Scheduled questions	`POST /projects/{id}/saved-questions`, `/dashboards`, `/scheduled-questions`	Dashboard tiles and recurring runs
9. Share	DBA, or a role with `share`	Share dialog	`POST /shares`	Read-only access for chosen users or roles
10. Open shared	Any viewer	Dashboards	`GET /shared-with-me`, `GET /dashboards/{id}/tiles/{tile_id}/result`	Each viewer's own result, never someone else's rows

The three loops that keep it accurate and safe

- **Teach:** a user's thumbs-up (`POST /messages/{id}/feedback`) creates a proposed SQL pair; a DBA reviews it in Knowledge and approves it; later questions retrieve it.
- **Stay in sync:** the worker profiles each project on its schedule; a drift report lists added, removed and changed tables and columns; the DBA starts a draft from the drift (`POST /drift-reports/{id}/create-draft`), edits it and redeploys, which returns to step 3.
- **Govern and measure:** every action writes an audit event that the audit browser filters and exports, and evaluation runs report accuracy, validity, latency and row-level security results against the targets in section 3.

3. Requirements

The plan has 32 functional and 12 non-functional requirements; each has a testable acceptance criterion that becomes a test or an evaluation case.

Functional requirements

ID	Requirement	Acceptance criterion
FR-01	Create a project that holds one PostgreSQL connection	The connection test returns server version and latency; credentials are stored encrypted; the test fails if the account can write; SSL can be turned on with an optional CA certificate; hosts outside the allowlist are refused
FR-02	Introspect the selected tables of a PostgreSQL database into a draft MDL	Schemas, tables, columns, types, primary and foreign keys, row counts and masked sample values are populated for the demo schema
FR-03	Draft model and column descriptions with the local LLM	Every model has a description; items are flagged "needs review"
FR-04	Validate and version the MDL on deploy	Invalid MDL is rejected with field paths; each deploy gets a version number; deploy is blocked if a model used by the golden set has no description
FR-05	Index a deployed MDL into Qdrant	Re-deploy removes stale chunks; retrieval returns only the active version
FR-06	Retrieve relevant models, columns, SQL pairs and instructions	At least 90% table recall on the golden set
FR-07	Classify intent (SQL, general, clarify, out of scope)	At least 95% correct on a 30-prompt intent set
FR-08	Rewrite follow-ups into standalone questions	A 5-turn conversation resolves references correctly
FR-09	Generate SQL over logical models as structured output	Output parses into `SQLCandidate` and references only deployed models, columns, calculated fields and metrics
FR-10	Compile logical SQL into PostgreSQL SQL	Calculated fields, metrics and relationship joins expand correctly on the compile test set; output references only deployed tables; physical table names in logical SQL are rejected

ID	Requirement	Acceptance criterion
FR-11	Validate compiled SQL	A single read-only `SELECT`, no unknown relations, a passing dry run; DML, DDL and system catalogs are rejected
FR-12	Auto-correct up to 3 times from typed errors	Correction rate and final success rate are reported by the evaluation run
FR-13	Execute read-only with limits	`DROP` and `UPDATE` prompts never reach the database; default limit 1000 rows; statement timeout enforced
FR-14	Stream progress via SSE	Ordered events end in `done` or `error`; the client sends the Authorization header; every step event carries its duration
FR-15	Record feedback as proposed SQL pairs; a DBA approves them and manages instructions	An approved pair is retrievable on the next similar question; a user's thumbs-up alone is not
FR-16	Log in with a local account and stay signed in with short-lived tokens	Login returns an access token and sets a refresh cookie; refresh rotates the cookie; logout revokes the session; a reused refresh token revokes the whole session
FR-17	Log in with LDAP	The API verifies the password by binding to the directory; the account is created on first login; directory groups map to roles; a removed group removes the role at the next login
FR-18	Log in with OIDC single sign-on	Authorization code flow with PKCE against a self-hosted provider; claims map to roles and user attributes; the resulting session is identical to a password login
FR-19	Built-in and custom roles	DBA and user are system roles that cannot be edited; a DBA can create, edit and delete custom roles from the capability list; deleting a role that is in use is refused

ID	Requirement	Acceptance criterion
FR-20	Enforce capabilities on every endpoint	Every DBA-only endpoint returns 403 for a user; a test fails if any route lacks a declared capability; role changes apply within a minute
FR-21	A DBA manages accounts and project access	Create, disable and re-role accounts; grant project access to users or roles; unlock accounts and issue temporary passwords; temporary password change at first login; the last active DBA cannot be disabled or demoted
FR-22	Row-level security	A DBA defines per-model policies from user attributes; the compiler injects them into every query; a missing attribute returns no rows; a DBA can preview a question as a chosen user; 0 rows outside policy across the row-level security suite
FR-23	Audit events and a full audit browser	Each security and change action writes an append-only event with actor and time; the browser filters by actor, action, target and date range, opens a detail view and exports CSV
FR-24	Visual modeling canvas	Models and relationships render as a diagram; a relationship is created by dragging between columns; descriptions and calculated fields are edited in a side panel; positions are saved with the draft; a failed compile check offers an Apply fix when a close name exists
FR-25	Metrics UI	A DBA defines dimensions and measures in a form; the UI shows the compiled SQL and a sample result before the metric is saved
FR-26	Scheduled profiling and drift detection	A schedule per project re-profiles the database; a drift report lists added, removed and changed tables, columns and types against the deployed model; a DBA can start a new draft from the drift in one action

ID	Requirement	Acceptance criterion
FR-27	Dashboards	A DBA pins an answer as a tile and arranges tiles; tiles refresh on demand or on a schedule; each viewer sees results computed under their own row policies
FR-28	Scheduled questions	A DBA saves a question with an hourly, daily, weekly or cron schedule; runs use the owner's permissions; the last 20 runs are kept; failures are recorded and shown; a schedule can be paused
FR-29	Sharing	A DBA shares dashboards, saved questions and threads with named users or roles as read-only; sharing can be revoked; a viewer never sees rows their own policy forbids
FR-30	Evaluation harness	One command runs the golden set, intent set, follow-up chains, red-team set and row-level security suite, and writes a report with a diff against the previous run
FR-31	Setup wizard: connect, select tables, define relationships	Three steps with a progress indicator; the connection is tested before it is saved; the table picker handles 200 or more tables with search and select-all; the relationship step lists foreign-key suggestions marked auto-generated and lets a DBA add, edit and delete; progress survives a browser restart; finishing creates a draft MDL and opens the canvas
FR-32	Sample projects	E-commerce and Human Resources samples create a ready project on the bundled demo database in one click, with tables and relationships preselected; a sample can be deleted

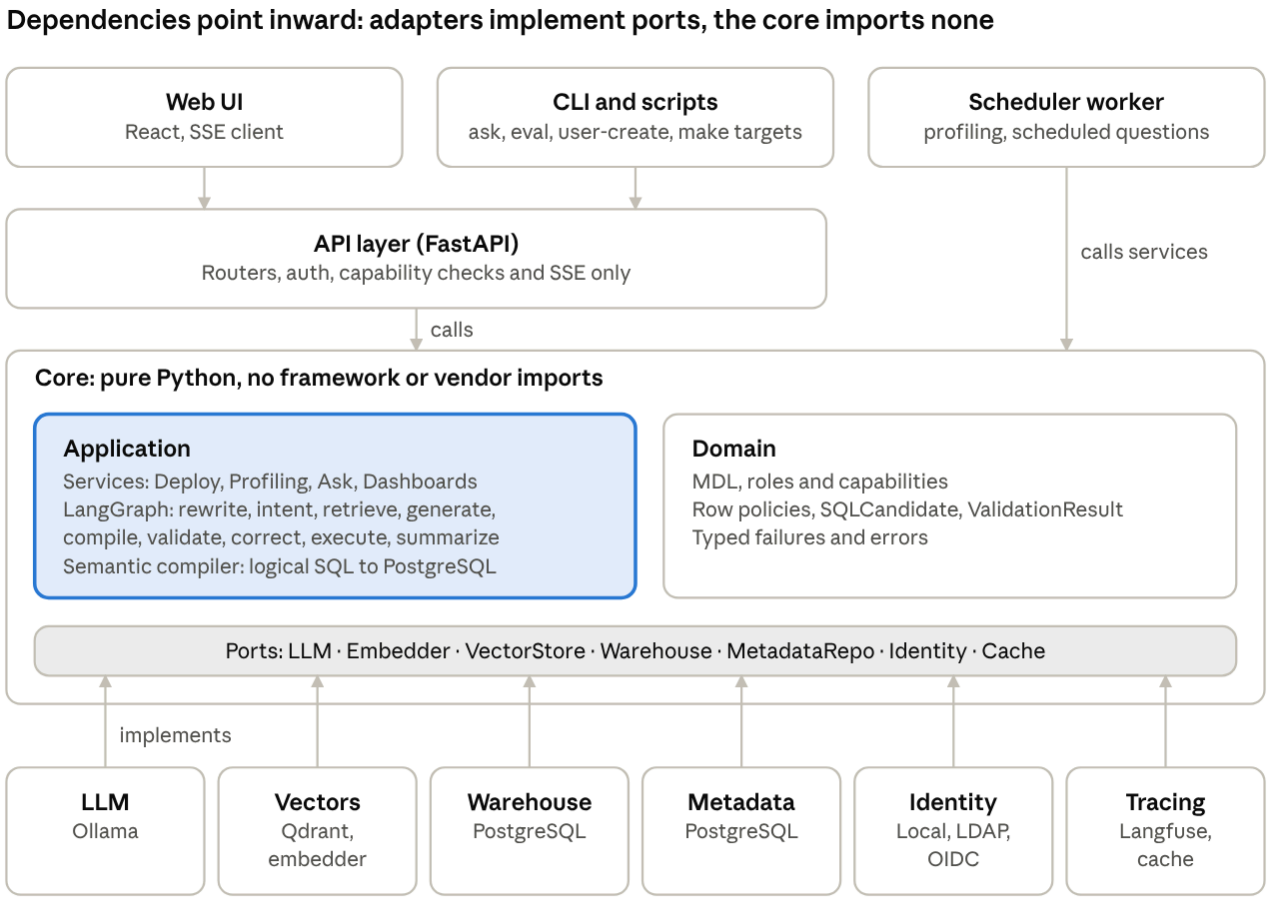
Non-functional requirements

ID	Requirement	Target
NFR-01	Execution accuracy on the golden set	Set on Day 3 from the chosen local model's baseline; provisional 70%, stretch 80%
NFR-02	Latency	p50 under 20 s, p95 under 45 s end to end on the reference hardware; first event under 2 s
NFR-03	Safety	0 non-SELECT statements executed across the red-team set
NFR-04	Tenant isolation	Every Qdrant query and metadata read filtered by project; cross-project tests for every endpoint
NFR-05	Observability	A trace per question with node timings, token counts and GPU time; a trace per scheduled job
NFR-06	Testability	Domain and application layers at 85% line coverage with fake adapters
NFR-07	Portability	One docker compose up; configuration only through environment variables; no outbound network call at runtime, proven by a test that blocks egress
NFR-08	Maintainability	Ruff and mypy clean; no adapter imports inside `domain` or `application` (enforced by import-linter)
NFR-09	Authentication hardening	argon2id hashes; access tokens valid 15 minutes and sessions 8 hours (provisional); 5 failed logins in 15 minutes lock the account for 15 minutes (provisional); no password or token in any log
NFR-10	Row-level security correctness	0 rows outside policy across the suite, including follow-ups, edited SQL, dashboards, scheduled runs, shared views and preview-as-user
NFR-11	Scheduler reliability	A job never runs twice for the same slot (single lease); missed and failed runs are recorded; a worker restart resumes the schedule

ID	Requirement	Target
NFR-12	Audit integrity	Audit events are append-only: the application database role has insert and select only on the audit table

4. System architecture

The service is a hexagonal core (domain, application, ports) wrapped by thin adapters, so the LLM, the vector store or the identity provider can be swapped without touching business logic.



Only adapters import ollama, qdrant_client, psycopg, sqlalchemy, ldap3 or authlib; import-linter enforces it.

hexagonal architecture · PostgreSQL only, 6 adapters

Requests enter through the API layer, and the scheduler worker enters through the same application services; both reach the outside world only through ports that adapters implement.

Request data flow

1. The UI logs the user in and posts a question to `/threads/{id}/ask` with a bearer token; the auth dependency resolves the principal: user, role, capabilities, project access and attributes.
2. `AskService` loads thread history and starts the graph with a checkpointer keyed by `thread_id`.
3. Nodes call ports: retrieval uses `EmbedderPort` and `VectorStorePort`, generation and correction use `LLMPort`, the compile node applies the semantic compiler and the principal's row policies, and execution uses `WarehousePort`.
4. Every node update is mapped to an SSE event and flushed to the client.
5. The final state is saved through `MetadataRepoPort`, an audit event is written and the trace is closed with tokens and GPU time.
6. Scheduled work (profiling, scheduled questions, dashboard refresh) runs in the worker and calls the same services with the owner's principal.

Tech stack

Layer	Choice	Notes
Runtime	Python 3.11+, `uv`	Lockfile committed
API	FastAPI, Pydantic v2, uvicorn	SSE through `StreamingResponse`
Agent	LangGraph `StateGraph` with a checkpointer	`MemorySaver` in dev, Postgres saver in the stack
Semantic compiler	sqlglot	Parses logical SQL, rewrites it to CTEs, injects row policies, validates; target dialect fixed to PostgreSQL
LLM	Local open-weight models, served by Ollama, behind `LLMPort`	Model names in config; llama.cpp or vLLM can replace Ollama through the same port
Vector store	Qdrant	Payload filters for tenancy and version; embeddings from a local model through the embedder port
Warehouse	PostgreSQL through psycopg	Read-only role; the only connector
Metadata store	PostgreSQL through SQLAlchemy 2 and Alembic	Tests use a Postgres container as well
Identity	argon2-cffi, PyJWT, ldap3, Authlib	Local accounts, LDAP bind, OIDC code flow with PKCE

Layer	Choice	Notes
Scheduler	APScheduler in a separate worker process	Jobs stored in the metadata database; a lease prevents double runs
Observability	Self-hosted Langfuse or structured logs only, Prometheus client	Behind an adapter
Quality	ruff, mypy, pytest, import-linter, Playwright	`make check` gate
Frontend	React, TypeScript, Tailwind, React Flow (canvas), Vega-Lite (charts), generated OpenAPI client	Vite build
Packaging	Docker, docker compose with an identity-demo profile, GitHub Actions	

Repository layout

```
text2sql/
  CLAUDE.md  Makefile  pyproject.toml  docker-compose.yml  .env.example
  .claude/            commands/  agents/  settings.json
  docs/               PROJECT.md  DECISIONS.md
  app/
    domain/           mdl.py  sql.py  errors.py  thread.py  auth.py
policy.py  schedule.py
    ports/            llm.py  embedder.py  vector_store.py  warehouse.py
metadata_repo.py  identity.py  cache.py
    application/
      services/       deploy.py  indexing.py  profiling.py  drift.py
retrieval.py  ask.py
                      auth.py  users.py  roles.py  policies.py  audit.py
                      dashboards.py  scheduled_questions.py  sharing.py
      semantic/       compiler.py  rls.py  validator.py
      agent/          state.py  graph.py  nodes/  prompts/
    adapters/         llm/  embedder/  vector/  warehouse/  metadata/
identity/ (local, ldap, oidc)  tracing/
    api/              main.py  routers/  schemas/  sse.py  deps.py
permissions.py
    worker/           main.py  jobs.py
    cli.py  config.py
  tests/              unit/  contract/  integration/
```

```
eval/{golden,intent,followups}/  redteam/  rls/
  ui/                 React app: screens for login, chat, model, knowledge,
dashboards, scheduled, users, audit, drift
  scripts/            db/grants.sql  demo/seed.sql  identity-demo/
  reports/            evaluation reports
```

Local runtime and hardware

Every model runs on your own hardware through Ollama (llama.cpp or vLLM can sit behind the same `LLMPort`); the sizes below are starting points that the Day 1 bake-off confirms or replaces.

Role	Starting candidates	Notes
SQL generation and correction	Qwen2.5-Coder 14B instruct; 32B if memory allows, 7B as the floor	In a June 2026 BIRD study, generation mattered more than raw size: Qwen2.5-Coder 7B scored 39.1 and Llama-3.3-70B scored 49.2 (study). BIRD is harder than a curated semantic layer, so read these as relative signals, not targets
Intent, rewrite, summary, chart spec	A 7B to 8B instruct model	Must follow a JSON schema reliably; keep it resident beside the SQL model if memory allows
Embeddings	`bge-m3`, `nomic-embed-text` or `mxbai-embed-large`; `all-MiniLM-L6-v2` on CPU-only machines	Choose on Day 3; the dimension is then fixed, and changing it forces a re-index
Avoid	Ollama models tagged `-cloud`; any model that fails the structured-output test	`-cloud` models run on Ollama's servers, not on your machine, and Ollama's cloud does not support structured outputs (docs)

Reference hardware tiers (memory figures are approximate for 4-bit quantization and exclude context memory)

Tier	Machine	Models that fit	What to expect
A	CPU only, 16 GB RAM	One 7B model for every role	Development and demos; the latency targets will not hold
B (reference)	GPU with 12 to 16 GB VRAM, or Apple Silicon with 24 to 32 GB unified memory	7B and 14B models	The tier the targets in this document assume
C	GPU with 24 GB VRAM or more, or 48 GB unified memory	A 32B model, or 14B plus 7B resident together	Headroom for correction loops
D	48 to 80 GB of GPU memory	A 70B-class model	Highest accuracy; check that the gain justifies the hardware

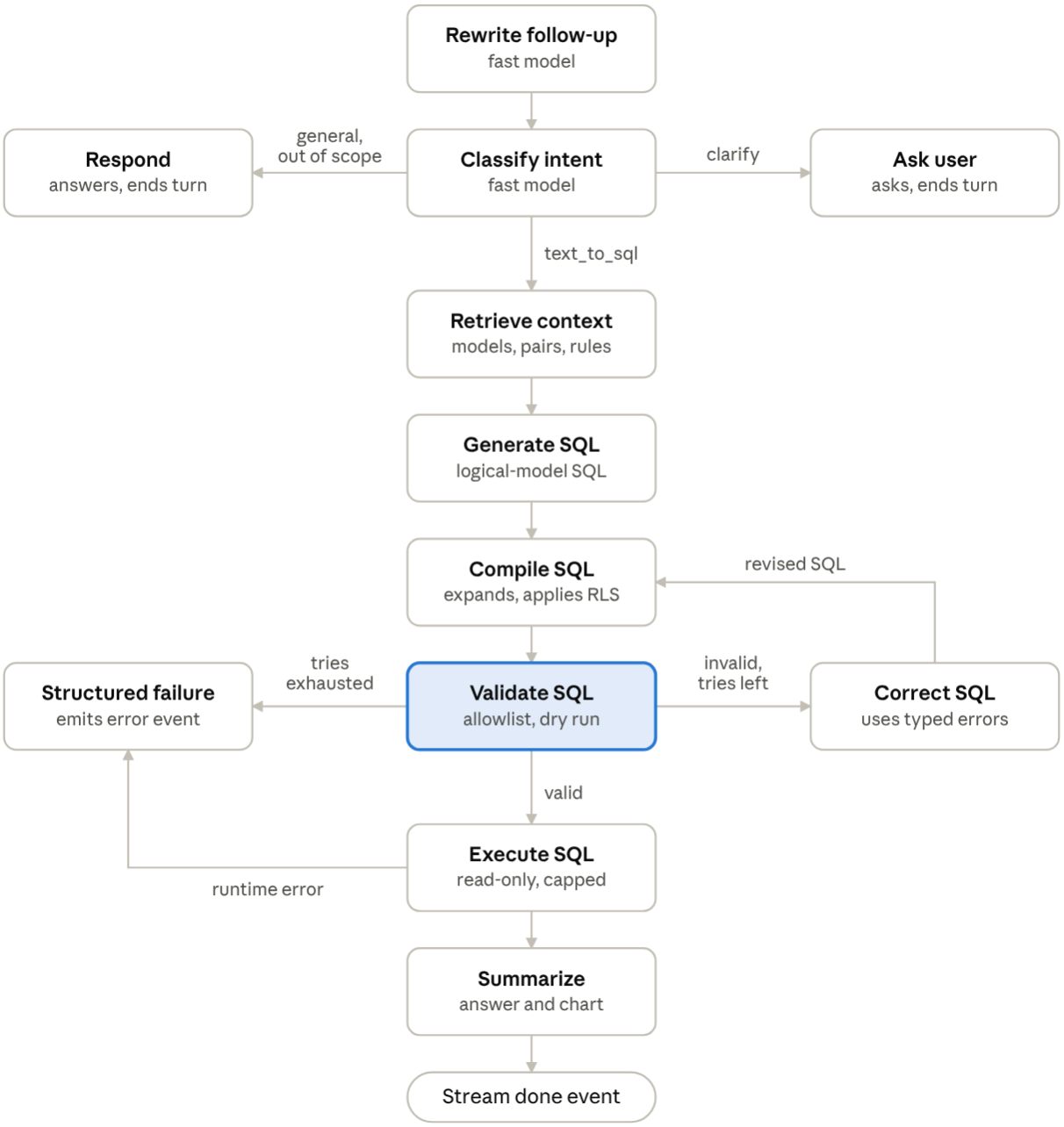
Offline and no-egress checklist

- Pull models and Python packages once on a connected machine, and record model digests in `docs/DECISIONS.md`.
- Run the compose stack on an internal network with no default route; an egress test fails the build if any container reaches the internet.
- Turn off telemetry in each tool (Qdrant, self-hosted Langfuse) and use offline mode for Hugging Face libraries; confirm the setting names in each tool's documentation.
- Point LDAP and OIDC at servers on your own network; the identity-demo profile ships OpenLDAP and Keycloak containers for testing only.
- Keep models loaded between requests (Ollama's `keep_alive` setting) so the first question of the day does not pay the load time.
- Check each model's license before you ship or share the stack.

5. LangGraph agent design

The agent is a LangGraph `StateGraph` whose only route to execution passes through the semantic compiler and the validator, with at most three typed corrections before it fails visibly.

Every SQL candidate is compiled and validated before it can execute



LangGraph agent · 12 nodes, 3 conditional branches

A question is rewritten and classified; SQL questions are retrieved, generated over logical models, compiled into PostgreSQL SQL under the user's row policies, validated, executed and summarized, while invalid SQL loops through correction until it passes or the attempts run out.

State

```
class AgentState(TypedDict, total=False):
    project_id: str
    thread_id: str
    principal: Principal              # user, role, capabilities, attributes
    question: str
    history: list[Message]
    standalone_question: str
    intent: Literal["text_to_sql", "general", "clarify", "out_of_scope"]
    context: RetrievedContext         # models, columns, relationships,
pairs, instructions
    logical_sql: str | None           # SQL over logical models, written by
the LLM
    sql: str | None                   # compiled PostgreSQL SQL, the only
SQL that can execute
    policy_context: str | None        # hash of the row policies applied to
this query
    validation_errors: list[ValidationError]
    attempts: int
    columns: list[str]
    rows: list[list]
    answer: str | None
    chart_spec: dict | None
    failure: StructuredFailure | None
```

Nodes

Node	Model	Reads and writes	Notes
`rewrite_followup`	Small	`question`, `history` to `standalone_question`	Pass-through when the thread is empty
`classify_intent`	Small	`standalone_question` to `intent`	Low confidence routes to `clarify`
`retrieve_context`	None	`standalone_question` to `context`	Hybrid search, relationship expansion, column pruning
`generate_sql`	SQL model	question and context to `logical_sql`	Structured output `SQLCandidate`; temperature 0; only logical model, column and metric names

Node	Model	Reads and writes	Notes
`compile_sql`	None	`logical_sql`, `principal` to `sql`, `policy_context`	Expands calculated fields, metrics and joins, injects row policies; compile errors become typed validation errors
`validate_sql`	None	`sql` and MDL to `validation_errors`	Parse, single `SELECT`, only compiler-emitted relations, dry run with `EXPLAIN`
`correct_sql`	SQL model	`logical_sql`, errors, context to `logical_sql`, `attempts` + 1	Quotes each typed error and lists the allowed logical names; the revised SQL goes back through the compiler
`execute_sql`	None	`sql` to `columns`, `rows`	Read-only role, timeout, row cap
`summarize`	Small	question, SQL, rows to `answer`, `chart_spec`	Streams tokens; chart spec checked against the real columns
`respond`	Small	question to `answer`	Non-SQL replies with no database access
`ask_user`	Small	question and context to `answer`	One clarifying question; ends the turn
`structured_failure`	None	errors to `failure`	Typed code, message and retryable flag

Conditional edges

- After `classify_intent`: `text_to_sql` goes to `retrieve_context`; `general` and `out_of_scope` go to `respond`; `clarify` goes to `ask_user`.
- After `validate_sql`: no errors goes to `execute_sql`; errors with `attempts` below `MAX_CORRECTIONS` go to `correct_sql`, which loops back through `compile_sql`; otherwise `structured_failure`.
- After `execute_sql`: success goes to `summarize`; a runtime error such as timeout or permission goes to `structured_failure`.

Prompt design rules

- Every LLM call returns a Pydantic model through structured output; no free-text parsing.
- The SQL prompt has fixed sections: role and PostgreSQL rules, instructions, relevant logical models rendered as DDL with descriptions, relationships, calculated fields and metrics, up to 3 approved SQL pairs, then the question; user text and data values are delimited as untrusted.
- The model never sees physical table names, policies or other users' attributes; it cannot reference what it was not shown, and the compiler rejects anything else.
- The correction prompt quotes each typed error and lists the allowed logical names; it never says only "try again".
- Prompts are versioned templates in `agent/prompts/`; the template version is recorded in each trace.

Rules for small local models

- Constrain each structured call with the JSON schema passed in Ollama's `format` field, repeat the schema text in the prompt, set temperature 0, and validate with Pydantic (Ollama docs). One repair retry on a parse failure comes before a failed attempt is counted.
- Keep prompts short: only the top-ranked models and columns, one-line descriptions, at most 3 SQL pairs. A 7B to 14B model loses the question in a long context faster than a hosted model does.
- If a model emits reasoning tokens, disable them for structured calls or strip them before parsing; reasoning never goes inside the structured fields.
- Put the static part of the prompt first and keep it byte-identical across calls so the server can reuse its cache.
- Keep the correction loop and quote the exact validator error: in an on-prem BIRD study, self-correction was the one technique that helped consistently at almost no cost (study).

Latency budget per stage (allocation of the 20 s p50 target on the reference hardware; targets to calibrate on Day 9, not measurements)

Stage	Budget (s)	Main lever if it overruns
Rewrite and classify	1.5	Merge both into one small-model call; keep both models loaded (Ollama `keep_alive`)
Retrieve context	0.5	Lower top-k, cache embeddings of repeated questions; keep the embedding model loaded

Stage	Budget (s)	Main lever if it overruns
Generate SQL	8.0	Shorter context, a smaller or more heavily quantized model, a stable prompt prefix the server can reuse
Compile	0.2	Cache compiled output per question, model version and policy context
Validate (including dry run)	0.5	Cache the dry-run result for identical SQL text; never skip it
Execute	2.0	Warehouse indexes, tighter row cap, query timeout
Summarize (streamed)	5.0	Small model, shorter answer, start streaming before the table renders
Total	17.7	2.3 seconds of headroom; each correction attempt adds a generate, a compile and a validate

The first event (`intent`) must reach the client within 2 s, so the small-model call stays first and nothing slow runs before it.

Persistence and streaming

- The checkpointer is keyed by `thread_id` (in-memory in development, Postgres in the stack); the `messages` table is the source of truth for the UI.
- Node updates stream to SSE events one to one, and a retry shows up as another `sql_candidate`, `compilation` and `validation` set rather than being hidden.
- A topology test asserts the node list and edges, so a change to the graph is always a deliberate diff.

6. Semantic layer, compiler and row policies

The MDL is the single source of truth that the LLM, the semantic compiler, the validator and the modeling canvas all share: if a name is not in the MDL, SQL that uses it is rejected. It is a versioned JSON document validated by Pydantic models in `domain/mdl.py`.

```
{
  "version": 3,
  "dialect": "postgresql",
  "models": [{
    "name": "orders",
    "tableReference": "public.orders",
```

```
    "description": "One row per customer order",
    "primaryKey": "order_id",
    "columns": [
      {"name": "order_id", "type": "integer", "isPrimaryKey": true},
      {"name": "customer_id", "type": "integer"},
      {"name": "region", "type": "text", "description": "Sales region of the
order"},
      {"name": "order_date", "type": "date"},
      {"name": "total_amount", "type": "numeric", "description": "Order value
in USD"},
      {"name": "discount", "type": "numeric"},
      {"name": "net_revenue", "type": "numeric", "isCalculated": true,
       "expression": "total_amount - discount"}
    ]
  }],
  "relationships": [{
    "name": "orders_customers", "models": ["orders", "customers"],
    "joinType": "MANY_TO_ONE", "condition": "orders.customer_id =
customers.id"
  }],
  "metrics": [{
    "name": "monthly_revenue", "baseObject": "orders",
    "dimensions": [{"name": "order_month", "expression": "date_trunc('month',
order_date)"}],
    "measures": [{"name": "revenue", "expression": "sum(net_revenue)"}]
  }],
  "instructions": ["Fiscal year starts on 1 April"],
  "layout": {"orders": {"x": 120, "y": 80}, "customers": {"x": 480, "y": 80}}
}
```

Validation rules (checked on deploy)

Rule	Failure behavior
Model, column, relationship and metric names are unique and match `[a-z][a-z0-9_]*`	Reject with field path
Every `tableReference` exists in the database and every non-calculated column exists on that table with a compatible type	Reject, list missing items
Every relationship references two existing models and a join condition that parses in PostgreSQL	Reject

Rule	Failure behavior
Calculated and metric expressions reference only columns of their own model and parse in PostgreSQL	Reject
Models without a description, or large tables with undescribed columns	Accept, flag "needs review", show in the deploy report

Metadata coverage gate. The deploy report shows the share of models and columns that have descriptions. A deploy is blocked when any model used by a golden-set question has no description, because missing descriptions are a common cause of wrong columns and wrong joins. Other gaps are flagged "needs review" and do not block.

How the MDL reaches the LLM. Retrieval picks relevant models and columns, and a relationship-aware step adds joined models. The prompt renders them as compact DDL-style text over logical names, with descriptions, relationship comments, calculated-field expressions and metric definitions. The LLM never sees physical table names.

The semantic compiler

The compiler turns SQL written over logical models into the only SQL that can reach the database. It is a pure function of the logical SQL, the deployed MDL, the principal and the row policies, and lives in `application/semantic/compiler.py`.

1. **Parse and restrict.** The logical SQL is parsed with sqlglot in the PostgreSQL dialect; anything other than a single `SELECT` (with optional `WITH`) is rejected as `NON_SELECT`.
2. **Resolve relations.** Every relation must be a deployed model or metric name, unqualified. Physical names, schema-qualified names, system catalogs and file-reading functions are rejected as `UNKNOWN_MODEL` or `DISALLOWED_FUNCTION`.
3. **Wrap each model.** Each referenced model becomes a CTE that selects only the columns the query uses, expands calculated fields as `expression AS name`, and applies the row policies of the principal as `WHERE` predicates with bound parameters. The reference in the query is replaced by the CTE, so a policy applies to every use of the model: joins, subqueries, set operations and aliases.
4. **Check joins.** A join between two models must match a declared relationship in either direction, otherwise `JOIN_NOT_ALLOWED`; a missing `ON` clause on a declared relationship is filled in.
5. **Expand metrics.** A metric referenced like a table becomes a grouped subquery over its base model with its dimensions and measures as ordinary columns.
6. **Emit.** PostgreSQL SQL is generated with a default `LIMIT` if none is present, together with the bound parameters and a `policy_context` hash.

7. **Report.** Any failure returns typed errors with the offending identifier and the closest allowed names, which feed the correction prompt.

Example: for a user whose `region` attribute is `EU`, the logical query

```
SELECT region, SUM(net_revenue) AS revenue FROM orders GROUP BY region
```

compiles to

```
WITH m_orders AS (
  SELECT region, (total_amount - discount) AS net_revenue
  FROM public.orders
  WHERE region = %(p_region)s            -- p_region = 'EU', bound as a
parameter
)
SELECT region, SUM(net_revenue) AS revenue FROM m_orders GROUP BY region
LIMIT 1000
```

The target dialect is a parameter of the compiler, fixed to PostgreSQL and tested only for PostgreSQL, because no other connector exists to run anything else.

Row-level security policies

Policies are stored in the `row_policies` table, not inside the MDL, so a change applies from the next question without a re-deploy.

```
{
  "name": "region_scope",
  "model": "orders",
  "predicate": "region = {{ user.region }}",
  "appliesTo": {"roles": ["user", "analyst_plus"], "exemptRoles": []},
  "onMissingAttribute": "deny"
}
```

Rule	Behavior
User attributes	Key-value pairs per account, set by a DBA or mapped from LDAP attributes and OIDC claims at login
Predicate template	A boolean SQL expression over the model's own columns; `{{ user.<attribute> }}` placeholders become bound parameters and are never concatenated into SQL; list attributes use `= ANY(...)`

Rule	Behavior
Combining	All policies that apply to a model combine with AND
Missing attribute	The predicate becomes false so no rows are returned, and an audit event is written
Exemptions	Listed per policy; no role is exempt by default, so a DBA also sees only permitted rows and tests policies with preview as user
Preview as user	A DBA can run a question as a chosen user to test a policy; the preview is labeled, audited and never stored on a dashboard
Save-time checks	The predicate must parse, reference only that model's columns and known attributes, and compile for a chosen test user
Caching	Compiled output and cached results are keyed by `policy_context`, the hash of the policies and attribute values that applied

What the canvas and the metrics UI edit. The modeling canvas edits the MDL draft: models are nodes, relationships are edges, a side panel edits descriptions, keys and calculated fields (each expression is checked live by the compiler), and node positions are stored in `layout`. The metrics UI edits `metrics`: pick a base model, add dimensions and measures, and see the compiled SQL and a sample result before saving.

7. Data design

Two stores carry state: a PostgreSQL metadata store for projects, models, accounts, policies, schedules and history, and Qdrant for everything that needs semantic retrieval. The application connects to the metadata store with a role that has insert and select only on `audit_events`.

Metadata store tables

Table	Key columns	Purpose
`projects`	id, display_name, connection_encrypted, ssl_mode, status, setup_step, is_sample, created_at	One per connected PostgreSQL database or sample dataset; setup_step lets the wizard resume
`mdl_versions`	project_id, version, mdl_json, status, deployed_at, deployed_by	Versioned models; one active per project; drafts have status draft
`threads`	id, project_id, user_id, title, created_at	Conversations, private to their owner
`messages`	id, thread_id, user_id, role, content, logical_sql, sql, policy_context, failure_json, trace_id, latency_ms, tokens, created_at	Turns with results and diagnostics
`sql_pairs`	id, project_id, question, logical_sql, status, proposed_by, approved_by, created_at	Status is proposed, approved or rejected; only approved pairs are indexed
`instructions`	id, project_id, text, is_default, created_at	Business rules
`users`	id, username, display_name, auth_source, password_hash, role_id, status, must_change_password, failed_attempts, locked_until, created_at, last_login_at	Accounts; auth_source is local, ldap or oidc; the hash is set for local accounts only
`user_attributes`	user_id, key, value, source	Attributes for row policies, set manually or mapped from LDAP and OIDC
`roles`	id, name, description, is_system	`dba` and `user` are system roles
`role_capabilities`	role_id, capability	A role is a set of capabilities
`identity_mappings`	id, source, match, role_id	Maps LDAP groups and OIDC claims to roles

Table	Key columns	Purpose
`project_members`	project_id, grantee_type, grantee_id, granted_by, granted_at	Projects a user or a role may use; an account's projects are the union of both
`sessions`	id, user_id, family_id, refresh_token_hash, expires_at, revoked_at, user_agent, created_at	Rotating refresh tokens; reuse of a rotated token revokes the family
`row_policies`	id, project_id, model, name, predicate, applies_to_json, on_missing, enabled, updated_by, updated_at	Row-level security policies
`audit_events`	id, actor_id, action, target_type, target_id, detail_json, ip, created_at	Append only
`profiling_schedules`	project_id, cron, enabled, last_run_at, last_status	Scheduled profiling per project
`drift_reports`	id, project_id, mdl_version, run_at, diff_json, status	Differences between the live schema and the deployed model
`saved_questions`	id, project_id, owner_id, question, logical_sql, created_at	Questions that can be scheduled, pinned or shared
`scheduled_questions`	id, saved_question_id, owner_id, schedule, enabled, next_run_at	Schedule for a saved question
`question_runs`	id, scheduled_question_id, started_at, finished_at, status, policy_context, result_ref, error	The last 20 runs per scheduled question are kept
`dashboards`	id, project_id, owner_id, title, layout_json, created_at	Dashboards
`dashboard_tiles`	id, dashboard_id, saved_question_id, chart_spec, position_json, refresh_schedule	Pinned answers

Table	Key columns	Purpose
`shares`	id, resource_type, resource_id, grantee_type, grantee_id, permission, granted_by, granted_at	Read-only grants to a user or a role
`result_cache`	resource_type, resource_id, policy_context, columns, rows, computed_at, expires_at	Cached results keyed by the policy context they were computed under
`job_leases`	job_key, holder, expires_at	One runner per job slot
`eval_runs` and `eval_results`	run_id, case_id, generated_sql, exec_match, latency_ms, gpu_ms	Evaluation history
project_tables	project_id, schema_name, table_name, model_name, selected, approx_rows	Tables chosen in the wizard's second step and the models created from them
app_settings	key, value_json, updated_by, updated_at	Installation settings such as the allowed database hosts

Safe sharing of results. A shared dashboard or saved question is never a copy of someone else's rows. A viewer's results are computed under the viewer's own policies, and a cached result is reused only when its `policy_context` equals the viewer's. A scheduled run stores its result under the owner's policy context, so another viewer triggers a live run instead of reading it.

Qdrant collections (one embedding model; dimension set in config; every point carries `project_id` and `mdl_version`)

Collection	Chunk	Extra payload	Retrieval
`db_schema`	One chunk per logical model (description, key, relationships, metrics) plus one per group of about 10 columns (name, type, description, masked sample values)	`model`, `kind`	Dense plus keyword, top 10 models, then expand joined models

Collection	Chunk	Extra payload	Retrieval
`sql_pairs`	Question text embedded; logical SQL in the payload; approved pairs only	`pair_id`	Top 3 by similarity, threshold 0.75
`instructions`	One rule per point	`is_default`	Defaults always included; others top 3
`question_history`	Successful standalone questions	`message_id`	Suggestions and cache hits

Row policies and user attributes are never indexed or shown to the LLM.

Indexing flow on deploy

1. Validate the MDL and write a new `mdl_versions` row as pending.
2. Build chunks, embed in batches, and upsert points tagged with the new version.
3. Mark the version active and delete points of older versions for that project.
4. If any step fails, keep the previous version active; deploy is all-or-nothing.

Retrieval strategy

- Filter every query by `project_id` and the active `mdl_version`.
- Prune columns when a model has more than about 50: keep key columns, columns matching the question, and calculated fields.
- Always include the relationship text for any two retrieved models that are directly joined.
- Log retrieved model names per question so retrieval recall can be measured separately from generation accuracy.

8. API specification

The API is REST plus one SSE stream, versioned under `/api/v1`, authenticated with bearer tokens, and limited to the projects and capabilities of the signed-in user. FastAPI generates the OpenAPI document; routers only translate HTTP to application services. Every route declares a capability (see the permission matrix below).

Method and path	Purpose	Notes
`POST /projects`	Create a project from the wizard's connection form (display name, host, port, username, password, database, SSL)	DBA; call after a successful test; credentials encrypted; rejected if the account can write
`POST /projects/{id}/connection/test`	Re-test a saved connection	Returns server version and latency
`POST /projects/{id}/introspect`	Draft MDL from the selected tables	Returns the draft and a profiling report
`PUT /projects/{id}/mdl`	Save a draft MDL, including canvas layout	Validation errors with field paths
`POST /projects/{id}/compile-preview`	Compile logical SQL for a chosen test user and return the SQL and a sample result	Used by the metrics UI, the canvas and the policy editor
`POST /projects/{id}/deploy`	Validate, index and activate	Returns the version and a deploy report
`GET /projects/{id}/mdl`	Active or draft MDL	`?version=` for older versions
`POST /projects/{id}/threads`	Create a thread	
`GET /threads/{id}`	Thread with messages	Owner, or a DBA
`POST /threads/{id}/ask`	Ask a question	`text/event-stream`; a DBA may pass `preview_as`
`POST /threads/{id}/rerun-sql`	Execute edited SQL	DBA only; same compiler, validator and limits
`POST /messages/{id}/feedback`	Thumbs up or down	A thumbs-up creates a proposed pair for DBA review
`GET/POST/DELETE /projects/{id}/sql-pairs`	Manage SQL pairs	DBA
`GET /projects/{id}/sql-pairs/proposed`, `POST /sql-pairs/{id}/approve` and `/reject`	Review proposed pairs	DBA; only approved pairs are indexed; each proposal includes the nearest approved pair

Method and path	Purpose	Notes
`GET/POST/DELETE /projects/{id}/instructions`	Manage instructions	DBA
`GET/POST/PUT/DELETE /projects/{id}/row-policies`	Manage row policies	DBA; saving compiles the policy for a test user
`GET/PUT /users/{id}/attributes`	Row-policy attributes of an account	DBA
`GET/PUT /projects/{id}/profiling-schedule`	Read or set the profiling schedule	DBA
`POST /projects/{id}/profile`	Run profiling now	DBA; idempotency key
`GET /projects/{id}/drift-reports`, `POST /drift-reports/{id}/acknowledge` and `/create-draft`	Review drift and start a draft from it	DBA
`GET/POST/DELETE /projects/{id}/saved-questions`	Saved questions	Owner
`GET/POST/PATCH/DELETE /dashboards`, `PUT /dashboards/{id}/tiles`, `POST /dashboards/{id}/refresh`	Dashboards and tiles	Build and edit need `create_dashboard`; open needs ownership or a share
`GET /dashboards/{id}/tiles/{tile_id}/result`	A tile's result	Always computed under the viewer's policies
`GET/POST/PATCH/DELETE /scheduled-questions`, `POST /scheduled-questions/{id}/run-now`, `/pause`, `/resume`, `GET /scheduled-questions/{id}/runs`	Scheduled questions and their run history	Needs `schedule_question`; the last 20 runs are kept
`GET/POST/DELETE /shares`, `GET /shared-with-me`	Share a dashboard, saved question or thread with a user or role	Needs `share`; read-only grants
`POST /projects/{id}/eval/run`, `GET /eval/{run_id}`	Start an evaluation and read its report	DBA; idempotency key
`POST /auth/login`	Sign in with a local or LDAP account	Returns an access token and sets the refresh cookie; rate limited
`GET /auth/oidc/start`, `GET /auth/oidc/callback`	Single sign-on	Authorization code flow with PKCE
`POST /auth/refresh`, `POST /auth/logout`, `POST /auth/change-password`	Session management	Refresh uses the cookie only; password change revokes other sessions

Method and path	Purpose	Notes
`GET /me`	Signed-in user, role, capabilities and projects	The UI builds navigation from this
`GET/POST/PATCH /users`, `PUT/DELETE /projects/{id}/members/{grantee_type}/{grantee_id}`	Accounts and project access	DBA; the last active DBA is protected
`GET/POST/PATCH/DELETE /roles`, `GET /capabilities`	Custom roles and the capability list	DBA; system roles are read-only
`GET/PUT /identity/mappings`	Map LDAP groups and OIDC claims to roles	DBA
`GET /audit-events`, `GET /audit-events/export.csv`	Audit browser data and export	DBA; filter by actor, action, target and date
`GET /health`, `GET /metrics`	Operations	`/health` includes the model server, Qdrant, database and worker
POST /connections/test	Test connection details from the wizard form without saving them	DBA; returns server version, latency and a precise failure reason; refuses writable accounts and hosts outside the allowlist
GET /projects/{id}/catalog/tables	List the database's tables for step 2	?search= and ?schema=; paged; schema, table, approximate rows and column count
PUT /projects/{id}/selected-tables	Save the tables chosen in step 2	Advances setup_step; removing a table that metrics, policies or calculated fields use is refused unless confirmed
GET /projects/{id}/relationship-suggestions	Suggested relationships for step 3	Foreign keys become auto-generated suggestions; column-name matches without a foreign key are marked inferred
PUT /projects/{id}/relationships	Save the reviewed relationships and finish setup	Builds or updates the draft MDL; setup_step becomes done
GET /projects/{id}/setup	Wizard state, so setup can resume	Current step, selected tables and pending suggestions

Method and path	Purpose	Notes
GET /samples, POST /samples/{id}/create	List and create the sample projects (E-commerce, Human Resources)	DBA; creates a ready project on the bundled demo database with tables and relationships preselected
POST /users/{id}/unlock, POST /users/{id}/reset-password	Unlock an account; issue a temporary password	DBA; a reset revokes the account's sessions and forces a password change at the next login
GET/PUT /settings/allowed-hosts	Read or edit the hosts the connection form may reach	Needs manage_connection; seeded from DB_HOST_ALLOWLIST; link-local and metadata addresses stay refused

Ask request

```
{ "question": "Top 5 customers by net revenue last quarter", "max_rows": 1000 }
```

SSE event contract (each event is `event: <type>` plus a JSON `data:` line, in this order; steps repeat on correction)

Event	Payload	When
`intent`	`{intent, standalone_question, used_context}`	After classification and rewrite
`retrieval`	`{models[], pairs[], instructions[]}`	After retrieval
`sql_candidate`	`{logical_sql, attempt}`	Each generation or correction
`compilation`	`{ok, errors[], sql?}`	After each compile; the compiled SQL is included for DBAs only
`validation`	`{ok, errors[{code, message, identifier?}]}`	After each validation
`correction`	`{attempt, reason}`	Before each retry
`rows`	`{columns[], rows[][], truncated, row_count}`	After execution
`answer_token`	`{text}`	Streamed summary tokens
`chart`	`{vega_lite_spec}`	Optional, after the answer

Event	Payload	When
`done`	`{message_id, latency_ms, tokens, gpu_ms}`	Terminal success
`error`	`{code, message, retryable}`	Terminal failure; the typed `StructuredFailure`
clarification	{question, options[]}	When the intent is clarify; the UI shows the options as quick replies
suggestions	{items[]}	After the answer: up to 3 follow-up suggestions from the small model

Users read the logical SQL, which uses business names; the compiled SQL contains physical names and policy predicates and is shown to DBAs only.

Every step event also carries `ms`, the time that step took, and the timeline shows each value and their sum. The UI shows error codes with an `E_` prefix, for example `E_UNKNOWN_COLUMN` and `E_TIMEOUT`.

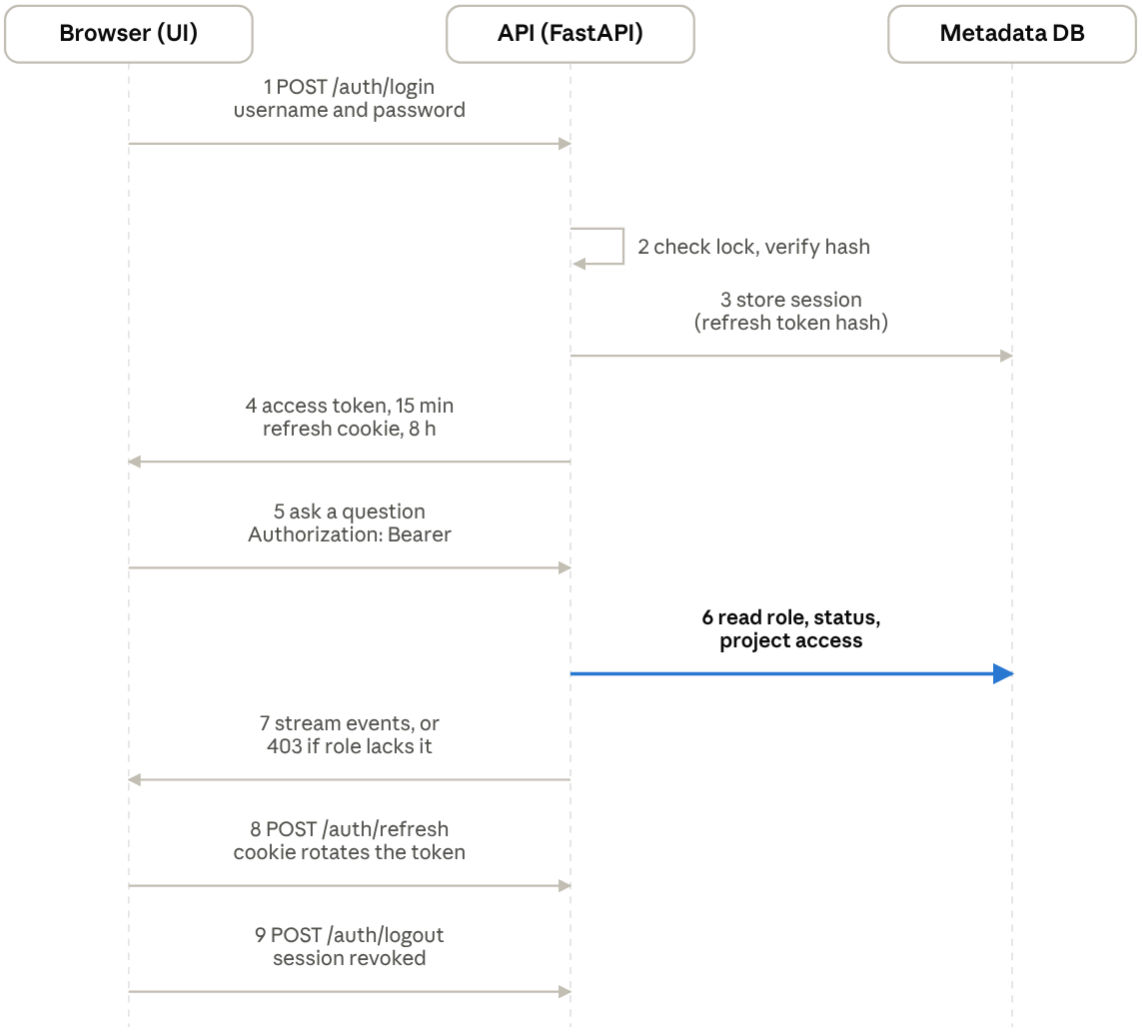
Error codes: `UNKNOWN_MODEL`, `UNKNOWN_COLUMN`, `JOIN_NOT_ALLOWED`, `NON_SELECT`, `DISALLOWED_FUNCTION`, `SYNTAX_ERROR`, `DRY_RUN_FAILED`, `TIMEOUT`, `EMPTY_RESULT`, `AMBIGUOUS_QUESTION`, `OUT_OF_SCOPE`, `LLM_UNAVAILABLE`, `UNAUTHENTICATED`, `FORBIDDEN`, `ACCOUNT_LOCKED`, `POLICY_ERROR`.

Conventions: request and trace IDs on every response, cursor pagination on lists, `409` when a deploy is already running for a project, and idempotency keys on deploy, profiling, run-now and evaluation runs.

Authentication, roles and permissions

Every person logs in with a local account, a directory account or single sign-on, and holds exactly one role: the built-in DBA or user role, or a custom role a DBA has built from capabilities.

Passwords are checked once; every request is re-checked for role and access



login and request flow · local account, 3 lanes, 9 steps

Credentials are verified once, by whichever login method applies; after that each request carries a short-lived access token, and the API re-checks the user's role, status and project access before it acts. The diagram shows a local account; LDAP and OIDC replace steps 1 to 3 and end in the same session.

Login methods

Method	How it works	Notes
Local	The password is verified against an argon2id hash	For accounts a DBA creates and for the bootstrap DBA

Method	How it works	Notes
LDAP	The API finds the user's entry with a read-only service bind, then binds as the user to verify the password; group membership maps to a role and directory attributes map to row-policy attributes	Uses `ldap3`; LDAPS or StartTLS required by default; the account is created on first login and no password is stored
OIDC single sign-on	The UI sends the browser to `/auth/oidc/start`; the API runs the authorization code flow with PKCE, state and nonce, validates the ID token (issuer, audience, signature from the provider's keys), and maps claims to a role and attributes	Uses Authlib; a self-hosted provider such as Keycloak on your own network

After any method the session is identical: the same access token, the same refresh cookie and the same checks on every request. Group and claim mappings are edited by a DBA in `identity_mappings`; they are re-applied at each login, so removing someone from a group removes the role at their next login.

Roles

Role	Capabilities	Notes
`dba`	All 19 capabilities below	System role, not editable; the last active DBA cannot be disabled or demoted
`user`	`ask_question`, `give_feedback`, `view_shared`	System role, not editable
Custom role	Any subset of the capabilities	Created, edited and deleted by a DBA; deletion is refused while accounts use the role

Capabilities

Capability	Grants
`ask_question`	Create threads and ask questions in assigned projects
`give_feedback`	Thumbs up or down on answers

Capability	Grants
`view_shared`	Open dashboards, saved questions and threads shared with the account, read-only
`view_all_threads`	Read other users' threads in a project
`rerun_sql`	See the compiled SQL, and edit and re-run generated SQL
`manage_connection`	Add, edit and remove database connections
`edit_mdl`	Introspect and edit the draft model, canvas and metrics
`deploy_mdl`	Deploy a model version
`manage_knowledge`	Add and edit SQL pairs and instructions
`approve_pairs`	Approve or reject proposed SQL pairs
`manage_policies`	Row policies and user attributes
`manage_profiling`	Profiling schedule, run now, drift actions
`create_dashboard`	Build and edit own dashboards and saved questions
`schedule_question`	Schedule saved questions
`share`	Share own resources with users or roles
`run_eval`	Run evaluations and read reports
`manage_users`	Accounts, project access and identity mappings
`manage_roles`	Create, edit and delete custom roles
`view_audit`	Read and export the audit log

Permission matrix (the custom-role column shows the capability that grants the action)

Action	DBA	User	Custom role
Log in, refresh, log out, change own password	Yes	Yes	Yes
Read own profile, capabilities and assigned projects (`GET /me`)	Yes	Yes	Yes
Ask questions and read own threads	Yes	Yes	`ask_question`
Give thumbs up or down	Yes	Yes	`give_feedback`

Action	DBA	User	Custom role
Open what is shared with the account	Yes	Yes	`view_shared`
Read other users' threads in a project	Yes	No	`view_all_threads`
See compiled SQL; edit and re-run SQL	Yes	No	`rerun_sql`
Add, edit or remove a database connection	Yes	No	`manage_connection`
Introspect, edit the model, canvas and metrics	Yes	No	`edit_mdl`
Deploy a model version	Yes	No	`deploy_mdl`
Add or edit SQL pairs and instructions	Yes	No	`manage_knowledge`
Approve or reject proposed SQL pairs	Yes	No	`approve_pairs`
Define row policies and user attributes	Yes	No	`manage_policies`
Set profiling schedules, run profiling, act on drift	Yes	No	`manage_profiling`
Build dashboards and saved questions	Yes	No	`create_dashboard`
Schedule questions	Yes	No	`schedule_question`
Share dashboards, questions and threads	Yes	No	`share`
Run evaluations and read reports	Yes	No	`run_eval`
Manage accounts, project access and identity mappings	Yes	No	`manage_users`
Create, edit and delete custom roles	Yes	No	`manage_roles`
Read and export the audit log	Yes	No	`view_audit`

Token and session rules (values are provisional defaults, set in configuration)

Item	Rule
Password storage	argon2id hash with a per-password salt; minimum length 12, no composition rules, and different from the temporary password; never logged; LDAP and OIDC accounts store no password
Login protection	The same generic error for a wrong user or a wrong password; 5 failed attempts in 15 minutes lock the account for 15 minutes; rate limits per IP address and per username

Item	Rule
Access token	Signed JWT holding the user id, a role hint, a token id and the expiry; valid 15 minutes; kept in browser memory, never in local storage
Refresh token	Random 256-bit value of which only the hash is stored; sent in an httpOnly, Secure, `SameSite=Strict` cookie limited to the auth path; valid 8 hours; rotated on every refresh
Reuse detection	Presenting an already-rotated refresh token revokes the whole session family and forces a new login
Role changes	The API reads role, capabilities and status from the database on every request (cached for under a minute), so a demoted or disabled user loses access within a minute rather than at token expiry
Logout and password change	Logout revokes the session; a password change revokes all other sessions
Transport	Serve over HTTPS through a local reverse proxy when the app is reachable from other machines; Secure cookies need HTTPS except on localhost
Streaming	The UI reads the ask stream with fetch streaming so it can send the Authorization header, which the browser EventSource cannot
CLI, CI and worker	They call the services directly and need no tokens; bearer tokens are for people using the HTTP API

Account lifecycle

1. First start: the first DBA is created from `BOOTSTRAP_DBA_USERNAME` and `BOOTSTRAP_DBA_PASSWORD`; remove both variables afterwards, and the account must change its password at first login.
2. A DBA creates local accounts with a temporary password and a role and grants project access; LDAP and OIDC accounts are created on first login with the role their mappings give. There is no self-signup. `make user-create` does the same from the command line.
3. A new or reset local account has `must_change_password` set and may only call the change-password endpoint until it is changed.

4. Disabling an account revokes its sessions at once and keeps its threads and audit history.
5. The last active DBA cannot be disabled or demoted; `make user-reset-dba` recovers an installation where every DBA is locked out.

A DBA can also unlock a locked account (`POST /users/{id}/unlock`) and issue a new temporary password (`POST /users/{id}/reset-password`); a reset revokes the account's sessions, sets `must_change_password` and is audited. LDAP and SSO accounts change their passwords in the directory or provider.

Enforcement

- Capabilities are rows in `role_capabilities` , seeded for the two system roles from `domain/auth.py` ; each router attaches a `require(capability)` dependency, and a second dependency checks project access against the account's memberships.
- A test walks every FastAPI route and fails if one has no declared capability (apart from `/health` , `/auth/login` , `/auth/oidc/*` and `/auth/refresh`), and a generated test calls every endpoint as a user and expects 403 wherever a capability is missing.
- The projects a request may touch come from `project_members` (every project for a DBA) and never from the request body (section 10).
- The UI hides what a role cannot use, but the API is the only authority.
- Unauthenticated calls return `401 UNAUTHENTICATED` , denied calls `403 FORBIDDEN` (and write an audit event), and locked accounts `423 ACCOUNT_LOCKED` .
- Authentication code lives in `domain/auth.py` (roles, capabilities, principal), `application/services/auth.py` , `users.py` and `roles.py` , `api/permissions.py` , and `adapters/identity/` (local, LDAP, OIDC), so the core stays free of crypto and directory libraries.

Audit events (appended to `audit_events`): successful and failed logins by method, logouts, password changes, account creation, disabling and role changes, custom role changes, identity mapping changes, project access changes, connection changes, model deploys, row policy changes, preview-as-user runs, SQL pair and instruction changes, proposal approvals and rejections, profiling and drift actions, dashboard, schedule and share changes, evaluation runs, and denied requests.

9. Frontend specification

The UI is a React app (Vite, TypeScript, Tailwind) that consumes the REST API through a generated client and reads the ask stream with fetch streaming. React Flow draws the modeling canvas and Vega-Lite draws charts. No business logic lives in the browser.

What each role sees. After login the UI calls `GET /me` and builds its navigation from the account's capabilities. A user sees Home (chat), what is shared with them, and their account menu. A DBA sees every screen below; a custom role sees the screens its capabilities unlock. The API enforces every limit; hiding a screen is a convenience, not a control.

Screen	Purpose	Key components	Capability
Login	Sign in with a password, a directory account or single sign-on; change a temporary password	Username and password form, a "Sign in with SSO" button, change-password form, silent token refresh, a session-expired notice that keeps the draft question	None
Home (chat)	Ask questions in threads and inspect every step	Thread list, composer, step timeline fed by SSE, SQL viewer (logical SQL for users; compiled SQL with edit and re-run for DBAs), results table, answer text, Vega-Lite chart, thumbs up and down, pin to dashboard, a "preview as user" picker for DBAs	`ask_question`
Connect and Model	Connect a database, introspect it and model it	Add database (opens the setup wizard), Connections with the allowed-hosts list, schema tree, edit tables and relationships, deploy report; a Canvas tab and a Metrics tab (below)	`manage_connection`, `edit_mdl`, `deploy_mdl`

Screen	Purpose	Key components	Capability
Canvas (tab)	Model the data visually	React Flow diagram: models as nodes with key columns, relationships as edges, drag between columns to create a relationship, side panel for descriptions, keys and calculated fields with a live compile check, auto layout, saved positions	`edit_mdl`
Metrics (tab)	Define metrics	Base model picker, dimension and measure editors, the compiled SQL and a sample result shown before saving	`edit_mdl`
Drift and profiling	Schedule profiling and review drift	Schedule form, run now, drift reports listing added, removed and changed tables, columns and types, and a "create draft from drift" action	`manage_profiling`
Knowledge	Teach the system	SQL pairs table, a queue of proposed pairs with a diff and approve or reject, instructions list	`manage_knowledge`, `approve_pairs`
Row policies	Define who sees which rows	Policies per model, predicate editor with an attribute picker, a test-user picker showing the compiled SQL and sample rows, attribute editor per account	`manage_policies`

Screen	Purpose	Key components	Capability
Dashboards	Build and open dashboards	Tile grid with drag and resize, tiles from saved questions, refresh, a "computed for you" label on each result, share dialog	`create_dashboard` to build, `view_shared` to open
Scheduled questions	Schedule and monitor saved questions	Schedule picker (hourly, daily, weekly, cron), run history with status and result, pause and resume, run now	`schedule_question`
Users and Roles	Accounts, roles and access	Accounts table with disable, unlock and reset-password actions, create-user dialog, custom role editor with capability checkboxes, identity mappings editor, project access picker for users and roles	`manage_users`, `manage_roles`
Audit	Review activity	Filter bar (actor, action, target, date range), table with paging, detail drawer, CSV export	`view_audit`
Evaluation	Run and read evaluations	Launcher and a report view with the diff against the previous run	`run_eval`

Screen	Purpose	Key components	Capability
Setup wizard	Add a PostgreSQL database in three steps, or start from a sample dataset	Step indicator (Connect, Select tables, Define relationships), a PostgreSQL source card and sample dataset cards, connection form with a Use SSL toggle, searchable table picker, per-table relationship editor	manage_connection, edit_mdl
Shared with me	Open what other people have shared with the account	List of dashboards, saved questions and threads with type, owner and shared date; a read-only banner that says each result shows only the viewer's own rows	view_shared

Setup wizard: adding a database

A DBA adds a database through a three-step wizard, or starts from a sample dataset. Only PostgreSQL is offered as a source.

Step	What the DBA does	API calls	Rules and states
Setup project	Chooses the PostgreSQL source, or a sample dataset (E-commerce or Human Resources)	`POST /samples/{id}/create` for a sample	A sample creates a ready project on the bundled demo database and opens the canvas

Step	What the DBA does	API calls	Rules and states
1. Connect	Fills Display name, Host, Port (default 5432), Username, Password (masked, with a show toggle), Database name and the Use SSL toggle, which reveals an optional CA certificate field; presses Next	`POST /connections/test`, then `POST /projects`	Next runs the test and shows the server version and latency, or a specific reason: host unreachable, authentication failed, database not found, SSL required, account can write, host not allowed. Nothing is saved until the test passes
2. Select tables	Sees "N table(s)", searches, ticks tables or selects all (which applies to the filtered list), presses Next	`GET /projects/{id}/catalog/tables`, `PUT /projects/{id}/selected-tables`, then `POST /projects/{id}/introspect`	Next stays disabled until at least one table is selected and shows how many are; the list stays responsive with 200 or more tables through server-side search and virtualized rows

Step	What the DBA does	API calls	Rules and states
3. Define relationships	Reviews one card per selected table with From (table.column), To (table.column) and Type; adds, edits or deletes relationships; presses Finish	`GET /projects/{id}/relationship-suggestions`, `PUT /projects/{id}/relationships`	Foreign keys appear as suggestions labelled auto-generated; column-name matches appear as inferred and stay off until the DBA accepts them; a table with none shows an empty state; types are one-to-one, one-to-many and many-to-one; Finish builds the draft MDL and opens the canvas

Resume and edit. The wizard's progress is stored in `setup_step`, so closing the browser does not lose it, and `GET /projects/{id}/setup` returns where to continue. "Edit tables" and "Edit relationships" on the Connect and Model screen reopen steps 2 and 3 for an existing project and keep the descriptions, calculated fields and metrics of models that did not change. Removing a table that a metric, a calculated field or a row policy uses asks for confirmation and lists what it would break.

Application shell and chat details

- **Header:** a project switcher showing the project name and its database (for example "Acme Commerce, acme_prod, PostgreSQL 16"), a local-only badge with the host name, a notifications bell (failed scheduled runs, new drift reports) and an account menu with Change password and Sign out.
- **Sidebar groups,** shown only for screens the account can open: Ask (Home, Shared with me), Work (Dashboards, Scheduled questions), Admin (Setup, Connect and model, Drift and profiling, Knowledge, Row policies) and System (Users and roles, Audit, Evaluation). The sidebar collapses to icons, and on a phone a bottom bar offers Home, Shared with me and Dashboards.

- **No access:** a page the account may not open shows a no-access page that says which capability to ask a DBA to add.
- **Chat states:** success, streaming (composer disabled), clarifying question with quick-reply options, out of scope, typed failure with a details toggle and Retry, empty result, and LLM unavailable.
- **Answer actions:** Copy, Re-run, a bar and line chart toggle where the columns allow it, Pin to dashboard, Schedule this question and Share; follow-up suggestion chips under the answer (for example "Break it down by region"); and a note "Used previous question as context" when a follow-up was rewritten.
- **Step timeline:** each step shows its duration and the total ("6 steps, 2.4 s"), and correction attempts show as "Attempt 2 of 3".
- **Themes:** light and dark.

Primary flow: ask a question

1. The user types a question; the composer disables and a step timeline appears.
2. Each SSE event adds or updates a step (intent, retrieval, SQL, compilation, validation, correction), so a retry is visible rather than hidden.
3. Rows render as soon as they arrive; the answer streams token by token; a chart appears if the backend emits one.
4. A DBA may edit the SQL and re-run it; any user can give thumbs up (proposes a SQL pair for DBA review) or thumbs down (flags the message for the next evaluation set); an account with `create_dashboard` can pin the answer to a dashboard.

UX rules

- Always show the SQL that produced a result; never present a number without its query.
- Show typed failures in plain language ("Column `order_dt` is not in the model; did you mean `order_date`?") with the raw error behind a details toggle.
- Label truncated results with the row limit, and label results that were computed under someone's preview.
- A dashboard tile shows the viewer's own result; if it is still loading, it says so instead of showing someone else's cached rows.
- Keyboard-first composer; empty states explain the next step (connect, deploy, then ask).
- Accessible by default: labelled controls, focus management on new messages, table semantics, canvas operations available from the keyboard.

Frontend acceptance criteria

- A DBA completes login, connect, deploy and first question in under 10 minutes using only the UI; a non-engineer does it with the demo script.
- A user never sees DBA screens in the navigation, and opening their URLs directly shows a no-access page.
- An expired access token is refreshed silently once; if the refresh fails, the UI returns to login and keeps the unsent question.
- A first-login account must change its password before any other screen loads; signing out in one tab ends the session in all tabs.
- On the canvas, a relationship drawn by dragging appears in the saved MDL, and a calculated field with a bad expression shows the compiler error before saving.
- Two viewers with different row policies open the same shared dashboard and see different, correct rows.
- Home, Shared with me and Dashboards work at 390 px; the other screens are designed for 1280 px, and the canvas falls back to a list editor on narrow screens.

UI design reference and alignment

The visual design (the Amazatic SQL Platform design file, with light and dark themes, a component sheet that shows every state, and a review bar to switch role, theme and viewport) is the reference for screens, states and wording. Where it differed from this document, the two were reconciled as follows.

Topic	Resolution
Administration role	There is no separate Admin role. The DBA sees Users and roles, Audit and Evaluation; separation of duties uses a custom role that holds only the administration capabilities
Row-policy exemptions	Listed per policy and empty by default; a DBA also sees only permitted rows and tests policies with preview as user
Project access	Granted to users and to roles; an account's projects are the union of both
Password rule	At least 12 characters, no composition rules, and different from the temporary password
Allowed hosts	A setting under Connections, edited by accounts with `manage_connection` and seeded from `DB_HOST_ALLOWLIST`; the wizard's "Host not allowed" message points to it

Topic	Resolution
Error codes	The API codes in section 8; the UI shows them with an `E_` prefix, for example `E_UNKNOWN_COLUMN` and `E_TIMEOUT`
Evaluation sets	The sizes in section 11 are minimums; the Evaluation screen reports the real counts of each suite
Themes	Light and dark, both defined as CSS variables, with a 2 px accent focus ring
Mobile	Home, Shared with me and Dashboards work at 390 px; other screens are desktop-first

10. Security, safety and governance

Safety is layered so that no single failure lets a harmful query run or exposes a row a person may not see: the database role, the semantic compiler, row policies, the SQL validator, execution limits, capabilities and tenant filters each stop a different class of problem.

Layer	Control	Verified by
Database	A dedicated read-only PostgreSQL role with `SELECT` on deployed tables only, read-only transactions and a statement timeout; the connection test refuses an account that can write	Integration tests attempt `DROP`, `UPDATE`, `COPY`, `CALL` and `pg_read_file` and expect permission errors
Semantic compiler	Only deployed logical models can be referenced; physical names, schema-qualified names, system catalogs and file-reading functions are rejected; joins must match declared relationships	Compile test set and the red-team set

Layer	Control	Verified by
Row-level security	Policies are injected by the compiler as bound-parameter predicates on every reference to a model; a missing attribute returns no rows; results are keyed by `policy_context`	The row-level security suite: 0 rows outside policy across aliases, subqueries, CTE shadowing, unions, set operations, window functions, metrics, follow-ups, edited SQL, dashboards, scheduled runs, shared views and preview-as-user
SQL validator	sqlglot allowlist on the compiled SQL: a single `SELECT`, only relations the compiler emitted, a passing dry run	Unit tests and the 30-prompt red-team set (injection, stacked statements, comment tricks, unknown tables)
Execution	Statement timeout (default 30 s), row cap (default 1000), result-size cap, one connection per request from a bounded pool	Tests with slow and large queries
Prompt injection	Question text and data values are delimited and labelled as untrusted; schema descriptions are data; the compiler and validator are the authority, and the model never sees policies or other users′ attributes	Red-team cases that embed "ignore previous instructions" in questions and in column descriptions
Authentication	Local accounts with argon2id, lockout, short-lived access tokens, rotating refresh tokens with reuse detection; LDAPS or StartTLS, bind verification; OIDC with PKCE, state, nonce and full ID-token validation	Tests for lockout, expiry, rotation and reuse; LDAP and OIDC integration tests against the demo containers
Authorization	Capability table with two system roles and custom roles; every route declares a capability; project access from memberships; role and status re-read on each request	Route-walk test, generated 403 tests, cross-project tests, demotion takes effect within a minute
Tenancy	`project_id` comes from the principal′s access, never from the request body, and is applied inside the Qdrant and metadata adapters	Cross-project tests for every adapter method

Layer	Control	Verified by
Sharing	Grants are read-only; every viewer's results are computed under the viewer's own policies; cached results are reused only for an identical policy context	Shared-dashboard tests with two viewers whose policies differ
Scheduler	Jobs run as the owner at run time: a disabled owner, or one who has lost a capability, makes the job fail closed and pause; a lease prevents double runs; jobs only execute saved questions through the full pipeline	Disabled-owner test, double-run test, worker-restart test
Secrets	Connection strings encrypted at rest (Fernet key from the environment); signing keys and bootstrap passwords only in environment variables; secrets scrubbed from logs and traces	Log-scan test in CI
Data exposure	Configured columns are masked in sample values and results; local models mean prompts and data never leave the host	Unit test on the masking function
Audit	`audit_events` is append-only: the application role can insert and select but not update or delete	Privilege test that attempts both
Connection input	Only accounts with manage_connection can submit connection details; the host must be inside the allowed-hosts setting (private ranges by default, seeded from DB_HOST_ALLOWLIST and edited in the app under Connections) and link-local and metadata addresses are always refused; short timeouts, no redirects; passwords are masked in the UI and never logged or echoed in errors	Tests with blocked addresses and a wrong-password case, plus a log scan

Rules the system must never break

- No statement other than a single read-only `SELECT`, compiled from logical models, reaches the database, whatever the question says.
- An unknown identifier is a failure, never silently fixed by guessing at execution time.
- A row outside a user's policy is never returned: not in an answer, a dashboard, a scheduled result, a share or a cached result.
- The LLM never receives credentials, policies, other users' attributes or sample values from masked columns.
- A failed question returns a typed failure rather than a plausible but unverified answer.

Local-only guarantees

- No outbound network at runtime: containers share an internal network with no default route, and a test fails the build if any container reaches the internet.
- Models and packages are pulled once, their digests recorded, and loaded from local storage; the application never downloads a model on demand.
- Telemetry is off in every component, and traces and logs stay on local disks.
- LDAP and OIDC servers are on your own network; the demo containers exist for testing only.
- If the host is shared, keep model files and the vector store on an encrypted disk.

Governance for the build itself

- Claude Code runs with permissions limited to the repo and a local development database; production credentials are never placed in the working directory.
- Hooks block commits that contain secrets and block edits to the files in `tests/redteam/` and `tests/rls/` unless a human approves.

11. Evaluation and quality plan

Quality is measured by comparing result sets, not SQL text: two different queries that return the same rows both count as correct. Evaluation starts on Day 3 and runs in CI from Day 9, so every prompt, retrieval or compiler change is judged by numbers. Security is measured the same way, with suites that must pass at 100%.

Test assets

- **Golden set:** 40 question and reference-SQL pairs from your real schema: 15 simple (single model), 15 join or aggregate, 10 hard (time logic, calculated fields, metrics, ambiguity). Each case stores the question, the reference logical SQL, expected models, an expected result checksum and a difficulty tag. Claude Code drafts candidates from the MDL; a DBA approves each reference before it enters the set.

- **Compile test set:** 40 logical queries with the expected compiled structure and results, covering calculated fields, metrics, joins, aliases and policies.
- **Intent set (30 prompts) and follow-up chains (10 chains of 2 to 3 turns).**
- **Red-team set:** 30 prompts and queries (injection, stacked statements, comment tricks, unknown tables, system catalogs, file functions).
- **Row-level security suite:** at least 60 cases across 3 test users with different attributes, run against every path that can produce rows: direct questions, follow-ups, edited SQL, aliases, subqueries, CTE shadowing, unions, window functions, metrics, dashboards, scheduled runs, shared views, cached results and preview-as-user.

Metrics and targets

Metric	Definition	Target
Execution accuracy	Compiled generated and reference queries return the same rows (order-insensitive unless the question asks for order)	Set on Day 3 from the baseline (provisional 70%, stretch 80%)
Retrieval recall	Expected models all appear in the retrieved context	90% or higher
Compile correctness	Compile test set results match the expectations	100%
Validity rate	First-attempt SQL passes compile and validation	70% or higher
Correction success	Failed first attempts fixed within 3 tries	60% or higher
Intent accuracy	Correct class on the intent set	95% or higher
Red-team pass	Malicious prompts blocked or neutralized	100%
Row-level security	Rows returned outside policy across the suite	0
Latency	p50 and p95 end to end on the reference hardware	under 20 s and under 45 s
Cost	Tokens and GPU seconds per question	Tracked; budget set after the first full run

Process

1. One command (`make eval`) runs every set against the real LLM and the demo database and writes a report (JSON and Markdown) plus a diff against the last run.
2. A cheap subset of 10 golden cases with a recorded LLM cache, plus the compile, red-team and row-level security suites (which need no LLM), runs on every pull request; the full run is nightly or on demand.
3. Failures are triaged by stage: retrieval miss, generation error, compiler gap, validation gap, or bad reference. Each fix adds or updates a case.
4. Thumbs-down messages in the UI are exported as candidate cases.

Observability

- One trace per question with a span for each LangGraph node, token counts, GPU time and the retrieved context (self-hosted Langfuse, or structured logs only, set through an adapter); one trace per scheduled job run.
- Structured JSON logs with request, thread and trace IDs; `/metrics` exposes request counts, node latencies, validation failures by code, correction counts, job runs and job failures.
- Dashboard views: accuracy trend by run, top failure codes, slowest nodes, scheduler health.

Accuracy levers, in the order to pull them (Day 8 and after)

1. **Metadata.** Fix descriptions, synonyms, sample values and relationships for the models in failing cases; it is the cheapest lever and improves retrieval and generation together.
2. **SQL pairs.** Add verified pairs for each failing question pattern; they also seed retrieval for similar questions.
3. **Instructions.** Encode business rules the model keeps missing: fiscal calendar, status codes, default filters, how to treat nulls and ties.
4. **Retrieval.** Tune top-k, keyword weight, relationship expansion and column pruning; check recall per case before touching prompts.
5. **Prompts and validator messages.** Sharpen PostgreSQL rules and make correction errors more specific. Self-correction was the one technique that helped consistently in an on-prem BIRD study, so protect that loop.
6. **Compiler coverage.** When the model keeps expanding a metric or a join by hand, add a rule or a metric so the compiler does it.
7. **Sampling and schema linking.** Generate several candidates and pick the one whose results agree, or add a schema-linking step before generation. Neither paid off in that study (self-consistency added about 0.13 points for roughly 5x the tokens, and a retrieval-based schema linker was no better than none), and local tokens are slow, so try them last and keep them only if your own evaluation shows a clear gain.

Change one lever per evaluation run so every accuracy delta has a single cause.

Test pyramid

Level	Scope	Uses	Runs
Unit	Domain, compiler, validator, graph nodes, permissions	Fake ports, no network	On every save
Contract	Each port against each adapter, including local, LDAP and OIDC identity	One shared suite per port	Every pull request
Integration	API, graph, Qdrant, metadata and demo PostgreSQL	`docker compose`	Every pull request (small set)
Identity integration	LDAP and OIDC logins and mappings	OpenLDAP and Keycloak from the identity-demo profile	Every pull request that touches identity code, and nightly
Security	Route-walk, generated 403 tests, red-team set, row-level security suite	Fake LLM where possible	Every pull request
End to end	DBA, user, custom role, LDAP user and OIDC user flows including dashboards, scheduled questions and sharing	Playwright	Before merging to main
Evaluation	Golden set, intent set, follow-up chains	Real LLM or recorded cache	Fast subset on pull requests, full run nightly or on demand
Offline	The whole stack with outbound traffic blocked	Compose on an internal network	Before release

Model bake-off (Day 1 on the 10-case subset, repeated on Day 3 with the full golden set)

Choose the smallest model combination that meets the targets on your hardware; larger models are the fallback, not the default.

1. Pull 2 or 3 SQL candidates, 1 or 2 small instruct models and 2 embedding models from the shortlist in section 4.
2. Run the same cases with identical prompts, temperature 0 and the same retrieved context for every model.

3. Record per model: execution accuracy, first-attempt validity, structured-output failures per 100 calls, p50 and p95 latency per node, peak memory and tokens per second.
4. Reject a model that fails more than 2 of 100 structured calls after one repair retry (provisional threshold), or that cannot stay resident beside the other models.
5. Write the choice, the model digests and the numbers to `docs/DECISIONS.md`, and rerun the bake-off when the hardware or the model changes.

What the evidence says: in a June 2026 study of on-prem open models on BIRD, self-correction helped consistently at almost no cost, a retrieval-based schema linker was statistically indistinguishable from no linking, and self-consistency added about 0.13 points for roughly 5x the tokens (study). BIRD schemas are small, so test retrieval and column pruning on your own large schemas before dropping them.

12. Claude Code operating model

Claude Code writes most of the code, but a plan this size only holds if three sessions can work at once without colliding, if every ticket is small and verifiable, and if review keeps up. The method below gives each track its own worktree and its own `CLAUDE.md` addendum, freezes the contracts between tracks on Day 1, and ends each day with merge gates.

Three parallel tracks

Track	Owns	Worktree and branch	Main outputs
A: semantic engine and agent	`domain/mdl.py`, `application/semantic/`, `application/agent/`, the deploy, indexing, profiling, retrieval and ask services, the LLM, vector and warehouse adapters, the evaluation harness	`../wt-a` on `track/a-engine`	MDL, introspection, retrieval, compiler, graph, validator, evaluation

Track	Owns	Worktree and branch	Main outputs
B: platform	`domain/auth.py`, `policy.py`, `schedule.py`, the auth, users, roles, policies, audit, drift, dashboards, scheduled-question and sharing services, the identity and metadata adapters, `worker/`, their API routers	`../wt-b` on `track/b-platform`	Accounts, LDAP and OIDC, roles, row policies, audit, scheduler, dashboards, sharing
C: web UI	`ui/`	`../wt-c` on `track/c-ui`	Every screen in section 9

Contracts that keep the tracks independent

- Ports and domain types are reviewed and frozen on Day 1; a change goes through a small pull request labelled `contract` that merges within the hour, and the other tracks rebase.
- The API is contract-first: the OpenAPI document is updated before the implementation, Track C builds against a mock server generated from it, and the real backend replaces the mock as each endpoint lands.
- Capability names (section 8) are fixed on Day 1 so the UI, the API and the tests agree.
- Alembic migrations form one linear history; each file is prefixed with its track and conflicts are resolved at the merge gate.

The loop for every ticket (1 to 3 hours each, in each track)

1. **Explore.** Start a fresh session in the track's worktree; ask Claude Code to read `CLAUDE.md`, the day's section of this document and the relevant files, and to summarize what exists. No code yet.
2. **Plan.** Switch to plan mode and ask for a plan for one ticket: files to touch, interfaces, tests to write, risks. Review and correct the plan; this is the cheapest place to catch a wrong design.
3. **Test first.** Ask for failing tests that encode the acceptance criteria (section 3), run them, and confirm they fail for the right reason.
4. **Implement.** Ask for the minimum code to pass, respecting the ports and adapters rules in `CLAUDE.md`.
5. **Verify.** Run `make check`; paste failures back and do not move on while red.
6. **Review.** Run the reviewer subagents on the diff, then read the diff yourself.
7. **Commit.** Small conventional commits; clear the context before the next ticket.

Daily rhythm

Time	Activity
09:00	Pull `main` into each worktree; run `make check` and `make eval-fast`; each track plans its tickets for the day in plan mode
09:30 to 12:30	The three tracks run in parallel; you rotate between sessions, reviewing plans and diffs as they land
12:30	Merge gate 1: merge finished tickets into `main` in the order A, B, C and run `make check`
14:00 to 16:30	Second block, starting with anything the gate found
16:30 to 17:30	Merge gate 2: integration run (`make check`, `make eval-fast`, security suites, Playwright smoke), reviewer subagents on the day's diff, fix list for tomorrow
17:30	`/day-wrapup` in each track: update the board and `docs/DECISIONS.md`, write tomorrow's first prompt

Review, not typing, is the bottleneck, so the plan budgets the gates and keeps tickets small enough to review in minutes.

Slip protocol. A ticket that is not done at gate 2 moves to the front of the next day on the same track. If a track is more than half a day behind, the other two tracks spend the first block of the next day reviewing and unblocking it instead of starting new tickets. Overflow is absorbed by the afternoon of Day 9 and by Day 10; nothing is removed from the plan.

Building blocks to set up on Day 1

Feature	What to configure	Why it matters here
`CLAUDE.md` (repo root)	Architecture rules, commands, conventions, definition of done, do-not-touch list (template in section 14)	Every session starts with the same constraints

Feature	What to configure	Why it matters here
Track addenda	Short `CLAUDE.md` files in `application/semantic/`, `application/agent/`, `worker/`, `ui/` and `tests/`	Local rules load only where they apply
Plan mode	Default for any ticket touching more than two files	Forces design review before code exists
Subagents (`.claude/agents/`)	The reviewers and helpers listed below	Separate context and a narrow brief give independent reviews
Slash commands (`.claude/commands/`)	`/ticket`, `/add-node`, `/add-adapter`, `/add-endpoint`, `/add-screen`, `/review-diff`, `/eval`, `/day-wrapup`	One-line, consistent workflows (library in section 14)
Hooks (`.claude/settings.json`)	After edits: ruff format and fast unit tests; before commit: `make check` and a secrets scan; block edits to `tests/redteam/` and `tests/rls/`	Quality is automatic
Permissions	Allow `uv`, `pytest`, `make`, `docker compose`, `npm`, non-force `git`; deny reads of `.env` and network tools against anything but localhost	Safe autonomy for long edit-test loops
MCP servers	Optional: a PostgreSQL MCP pointed at the demo database, GitHub for issues and pull requests	Lets Claude Code inspect real schema and data
Git worktrees	The three above, created on Day 1	Parallel sessions without edit collisions
Headless mode (`claude -p`)	Scripted jobs: draft golden-set candidates, summarize evaluation diffs, generate the API client	Same tooling in CI and scripts

Subagent roster

Subagent	Brief	Used
`architect-reviewer`	Checks hexagonal boundaries, ports, naming and dependency injection	After every feature
`sql-safety-reviewer`	Hunts for ways SQL or prompts could bypass the compiler or validator	After Days 4, 5 and 7
`rls-reviewer`	Tries to construct queries, shares, caches and schedules that return rows outside a policy	After Days 5, 7 and 8
`auth-reviewer`	Reviews login, tokens, LDAP and OIDC flows, capability checks and session handling	After Days 2, 6 and 8
`ui-reviewer`	Checks role-based visibility, accessibility and error states	After Days 5, 7 and 8
`test-writer`	Writes failing tests from an acceptance criterion and never touches production code	Step 3 of the loop
`eval-analyst`	Reads an evaluation report, clusters failures by stage and proposes the smallest fix	Days 3, 6, 8 and 9

Context hygiene

- One ticket per session; use `/clear` between tickets and `/compact` only mid-ticket.
- Point Claude Code at files and at section numbers of this document instead of pasting long text.
- When Claude Code repeats a mistake twice, fix the rule in `CLAUDE.md` rather than correcting it again in chat.
- Keep `docs/DECISIONS.md`; ask Claude Code to append one line whenever a design choice changes.

What stays with you (do not delegate)

- Approving reference SQL for the golden set, the red-team prompts and the row-level security cases.
- The read-only database grants, identity provider settings and any credentials.
- The default capability sets of the two system roles and the meaning of every row policy predicate.
- Final review of the compiler, validator, token handling and prompts, because that is where accuracy and safety are decided.
- Judging whether evaluation failures are model errors, retrieval gaps, compiler gaps or wrong references.

13. Day-by-day execution plan

Every day has three tracks that run in parallel (section 12): A builds the semantic engine and agent, B builds the platform, and C builds the web UI. Each track has tickets, one primary prompt and a shared exit criterion you can check in minutes, and the day ends with a merge gate. Tickets reference the requirement IDs in section 3. If a day's exit criterion is not met at gate 2, the open tickets move to the front of the next day on the same track.

Safety is proven by Day 5 and accuracy by Day 9, with Day 10 as buffer

Three tracks run in parallel; a diamond marks an exit gate that must pass first



10-day plan · 3 parallel tracks, 3 exit gates

Safety is proven by the end of Day 5 and the backend contract by Day 7; accuracy is judged on Day 9, which leaves Day 10 as buffer. The evaluation bar runs from Day 3 so every change is measured.

Day 1: Foundations in parallel

Goal: a green skeleton where every external system sits behind a port, a seeded demo database, a model bake-off, and a UI shell running against a mock API.

Track A (engine and agent)

- [ ] D1-A1 Scaffold the repo with `uv`, ruff, mypy, pytest, pre-commit, import-linter and a Makefile; add the `CLAUDE.md` files, `.claude/` commands, agents and hooks; create the three worktrees
- [ ] D1-A2 Define the ports as Protocols and the domain errors (`LLMPort`, `EmbedderPort`, `VectorStorePort`, `WarehousePort`, `MetadataRepoPort`, `IdentityPort`, `CachePort`)
- [ ] D1-A3 PostgreSQL `WarehousePort` adapter with psycopg: read-only role, statement timeout, row cap, contract tests (FR-01, FR-13)
- [ ] D1-A4 Install Ollama, pull the shortlisted models and run the bake-off on a 10-case subset (section 11)

Track B (platform)

- [ ] D1-B1 Config, structured logging, `/health`; docker compose with Qdrant, the metadata and demo PostgreSQL databases, Ollama and a worker placeholder; seed an e-commerce demo schema of about 12 tables and a small human resources schema for the sample datasets
- [ ] D1-B2 Alembic migrations for projects, MDL versions, users, roles, role capabilities, sessions and audit events, with the insert-and-select-only grant on the audit table (NFR-12)
- [ ] D1-B3 CI workflow running `make check`, and the skeleton of the egress test

Track C (web UI)

- [ ] D1-C1 Scaffold Vite, React, TypeScript, Tailwind, React Flow, Vega-Lite, Vitest and Playwright
- [ ] D1-C2 Draft the OpenAPI contract, generate the TypeScript client and a mock server
- [ ] D1-C3 Layout shell, theming and route guards driven by capabilities (stubbed)

Primary prompts

- A: "Read docs/PROJECT.md sections 4, 10 and 12. Enter plan mode and propose the repo scaffold, the seven port Protocols and the import-linter contracts. After I approve, implement D1-A1 to D1-A3 with tests."

- B: "From section 7 write the Alembic migrations for the Day 1 tables. Grant the app role insert and select only on audit_events and add a test proving update and delete fail."
- C: "From the endpoint table in section 8 draft openapi.yaml for auth, me, projects, mdl and threads, generate the TypeScript client and a mock server, then build the layout shell with capability-based route guards."

Exit: `make check` is green; the demo schema is seeded; `SELECT 1` runs through the Postgres adapter; the audit update test passes; the chosen models answer a schema-constrained test prompt; the UI shell runs against the mock API. **Human checkpoint:** create the read-only database role (`scripts/db/grants.sql`), confirm the hardware tier and name the second reviewer.

Day 2: Introspection, the semantic model and local login

Goal: connect a PostgreSQL database and get a deployable draft MDL, and log in with a local account (FR-01 to FR-04, FR-16, FR-21, FR-31).

Track A

- [ ] D2-A1 `domain/mdl.py` with Pydantic models and the validation rules; one failing MDL fixture per rule
- [ ] D2-A2 `ProfilingService` on `pg_catalog` and `information_schema` : schemas, tables, columns, types, primary and foreign keys, row counts, masked sample values
- [ ] D2-A3 Local-LLM drafted descriptions with a `needs_review` flag
- [ ] D2-A4 Metadata repo adapter and the wizard endpoints: test connection, create project, list catalog tables, save selected tables, introspect them, save and read the MDL, wizard state
- [ ] D2-A5 Relationship suggestions for the wizard: foreign keys as auto-generated suggestions, column-name matches as inferred; save the reviewed relationships and finish setup

Track B

- [ ] D2-B1 `AuthService` : argon2id, login, refresh rotation, logout, change-password, sessions
- [ ] D2-B2 Lockout, rate limits, generic errors, the first DBA from environment variables with a forced password change
- [ ] D2-B3 `make user-create` and `make user-reset-dba` ; audit events for authentication actions

Track C

- [ ] D2-C1 Login and change-password screens, access token kept in memory, one silent refresh

- [ ] D2-C2 Setup wizard from section 9 (Connect with Use SSL, searchable table picker with select-all, relationship editor with auto-generated suggestions), then the Connect and Model screen with schema tree and description editing

Primary prompts

- A: "Implement D2-A1 from section 6 with Pydantic v2; write one invalid-MDL test per rule first. Then ProfilingService against the WarehousePort using only pg_catalog and information_schema, no vendor code in the service."
- B: "Implement AuthService per section 8. Write tests first for lockout, refresh rotation, reuse detection and password change revoking other sessions."
- C: "Build Login and Change password with the generated client, then the three-step setup wizard from section 9 against the mock API. Keep the access token in memory, retry once through /auth/refresh on 401, then return to login keeping the unsent input."

Exit: the wizard (connect, select tables, define relationships) creates a draft MDL from the demo database, and edit, save and read of an MDL work; the login, rotation and reuse tests pass; the UI logs in against the real API. **Human checkpoint:** read and correct the descriptions of the 5 most important tables; they drive accuracy.

Day 3: Indexing, retrieval, roles and the first canvas

Goal: deploy an MDL and retrieve the right context, enforce capabilities with custom roles, and see the model on a canvas (FR-05, FR-06, FR-19, FR-20, FR-24, FR-32).

Track A

- [ ] D3-A1 `EmbedderPort` and Qdrant adapter with project and version filters; contract tests including cross-project
- [ ] D3-A2 MDL chunker, `IndexingService`, all-or-nothing `DeployService`, `POST /deploy`
- [ ] D3-A3 `RetrievalService`: hybrid search, relationship expansion, column pruning
- [ ] D3-A4 Golden set v1 (40 cases drafted by Claude Code and approved by a DBA), compile test set v0 and an evaluation runner for retrieval recall
- [ ] D3-A5 Repeat the bake-off on the full golden set, choose the SQL, small and embedding models, and set the accuracy target from the baseline

Track B

- [ ] D3-B1 Roles, role capabilities and the two system roles seeded; custom role create, edit and delete with in-use deletion refused
- [ ] D3-B2 `require(capability)` and project-access dependencies, the permission cache (under one minute), the route-walk test and generated 403 tests
- [ ] D3-B3 Account and project-access endpoints (for users and roles); user attributes

- [ ] D3-B4 Sample projects: create the E-commerce and Human Resources samples from `POST /samples/{id}/create` with tables and relationships preselected

Track C

- [ ] D3-C1 Modeling canvas v1: models as nodes, relationships as edges, auto layout, saved positions
- [ ] D3-C2 Application shell (header with project switcher, notifications bell and account menu; sidebar groups Ask, Work, Admin and System; mobile bottom navigation for Home, Shared and Dashboards) and route guards driven by `GET /me` capabilities

Primary prompts

- A: "Implement IndexingService per section 7: build chunks, embed in batches, upsert with the new version, then activate and delete old versions; on any failure keep the old version active. Test failure injection at each step."
- B: "Implement roles and capabilities per section 8: system roles seeded from domain/auth.py, custom roles in the database, a require(capability) dependency, and a test that walks every route and fails if one lacks a capability."
- C: "Build the modeling canvas with React Flow: models as nodes showing key columns, relationships as edges, auto layout, positions saved into the MDL layout. Keep it keyboard accessible."

Exit: retrieval recall is 90% or higher on the golden set; re-deploy leaves no stale points; the role matrix tests pass for DBA, user and one custom role; the canvas renders the demo model and saves positions. **Human checkpoint:** approve or reject every golden reference query and record difficulty tags.

Day 4: The semantic compiler, audit and row policies

Goal: compile logical SQL into PostgreSQL SQL, record every action, and model row policies (FR-10, FR-23, FR-24, FR-25 and the model of FR-22).

Track A

- [ ] D4-A1 Compiler core: parse with sqlglot, resolve relations to deployed models only, one CTE per model, calculated fields, joins checked against declared relationships
- [ ] D4-A2 Metrics expansion into grouped subqueries
- [ ] D4-A3 Compile test set v1 (40 cases) with expected compiled results
- [ ] D4-A4 Render logical models as DDL-style prompt context
- [ ] D4-A5 `POST /compile-preview`

Track B

- [ ] D4-B1 Audit service: append-only events from every service, and the audit API with filters and paging
- [ ] D4-B2 Row policy domain and storage: policies, predicate templates, exemptions, user attributes, admin API
- [ ] D4-B3 Policy validation at save: the predicate parses, uses only the model's columns and known attributes, and compiles for a test user

Track C

- [ ] D4-C1 Canvas v2: create relationships by dragging, side panel for descriptions, keys and calculated fields with a live compile check
- [ ] D4-C2 Metrics UI: base model, dimensions, measures, compiled SQL and a sample result
- [ ] D4-C3 Deploy report view

Primary prompts

- A: "Implement application/semantic/compiler.py per section 6: parse with sqlglot in the PostgreSQL dialect, resolve relations only to deployed models and metrics, wrap each model in a CTE with its calculated fields, enforce joins against declared relationships, and return typed errors. Make the compile test set pass; list any case you could not make pass instead of editing it."
- B: "Implement the row policy model and API per section 6 with predicate templates, bound parameters and exemptions. Saving a policy must compile it for a chosen test user. Every policy change writes an audit event."
- C: "Add relationship creation by dragging on the canvas and the metrics form. Both call compile-preview and show the compiled SQL and a sample result; a failing expression shows the compiler error inline."

Exit: the compiler passes every compile case; policies are stored and validated; every action so far writes an audit event; canvas edits persist; the metrics UI previews compiled SQL. **Human checkpoint:** define the 3 test users and their attributes for the row-level security suite, and approve the policy predicates.

Day 5: Core graph, row-level enforcement and LDAP

Goal: a question becomes logical SQL, compiled SQL, validated SQL and rows from the command line, with row policies enforced and safety proven (FR-09, FR-11, FR-12, FR-13, FR-17, FR-22).

Track A

- [ ] D5-A1 `AgentState` and the graph: retrieve, generate (structured logical SQL), compile, validate, execute; compile errors become typed validation errors

- [ ] D5-A2 Validator on the compiled SQL: parse, single `SELECT`, only compiler-emitted relations, dry run with `EXPLAIN`
- [ ] D5-A3 Typed failures and the correction loop (3 attempts) back through the compiler; prompts tuned for small models
- [ ] D5-A4 CLI `ask` and evaluation runner v1 (execution accuracy); record the baseline

Track B

- [ ] D5-B1 Policy injection in the compiler (with Track A): filtered CTEs from the principal's attributes and the `policy_context` hash
- [ ] D5-B2 Red-team set (30 prompts) and the row-level security suite (60 or more cases, 3 test users) as parameterized tests
- [ ] D5-B3 LDAP adapter: service bind, user bind, group-to-role mapping, attribute mapping, tests against the OpenLDAP container

Track C

- [ ] D5-C1 Chat screen: composer, step timeline fed by fetch-streamed SSE, results table
- [ ] D5-C2 SQL viewer with copy and re-run, bar and line chart toggle, follow-up chips and clarification quick replies: logical SQL for users, compiled SQL for accounts with `rerun_sql`

Primary prompts

- A: "Build the graph per section 5 with a fake LLM first: retrieve, generate, compile, validate, execute, with the correction loop capped at 3. Write the topology test. Then implement the validator as a pure function (compiled sql, mdl) to ValidationResult and make every red-team case pass without changing the cases."
- B: "Wire row policies into the compiler: wrap each model reference in a filtered CTE with bound parameters, fail closed on a missing attribute, and return the policy_context hash. Then write the row-level security suite and report any query shape that leaks instead of weakening a case."
- C: "Build the chat screen with fetch streaming. Render each SSE event as a step, show rows as they arrive, and show the compiled SQL only when the account has rerun_sql."

Exit (safety gate): the red-team set and the row-level security suite pass at 100%; the baseline accuracy is recorded; LDAP login works against the OpenLDAP container; the `sql-safety-reviewer` and `rls-reviewer` have no blocking findings. **Human checkpoint:** review the compiler and validator line by line, and try 10 hostile queries as a user with a restrictive policy.

Day 6: Conversation, teaching, single sign-on and the scheduler

Goal: multi-turn threads, the teaching loop, OIDC login, and a worker that profiles on a schedule and reports drift (FR-07, FR-08, FR-15, FR-18, FR-26).

Track A

- [ ] D6-A1 `rewrite_followup` and `classify_intent` nodes with the clarify and respond paths; thread persistence
- [ ] D6-A2 Instructions and approved SQL pairs in retrieval and prompts; the proposal flow from thumbs-up to proposed to approved
- [ ] D6-A3 `summarize` and chart nodes with Vega-Lite validation against the real columns
- [ ] D6-A4 Intent set (30 prompts) and follow-up chains (10)

Track B

- [ ] D6-B1 OIDC adapter with Authlib: PKCE, state, nonce, ID-token validation, claim-to-role and attribute mappings, tested against Keycloak
- [ ] D6-B2 Worker process: APScheduler with jobs in the metadata database, leases, resume after restart
- [ ] D6-B3 Scheduled profiling and drift detection: live catalog against the deployed MDL into `drift_reports`, plus create-draft-from-drift

Track C

- [ ] D6-C1 Knowledge screen: SQL pairs, instructions, and the proposed-pairs queue with the nearest approved pair beside each proposal and approve or reject
- [ ] D6-C2 Users and Roles screens: accounts with unlock and reset-password actions, custom role editor, identity mappings, project access for users and roles
- [ ] D6-C3 Row policies screen with a test-user preview

Primary prompts

- A: "Add rewrite_followup and classify_intent as the first two nodes, both on the small model with structured output; update the topology test. The rewritten question must stand alone; test pronouns, ellipsis and a topic switch."
- B: "Implement the OIDC adapter per section 8 with Authlib: authorization code flow with PKCE, state and nonce, full ID-token validation, then issue the same local session as a password login. Test against the Keycloak container, including a removed group at the next login."
- C: "Build the Users and Roles screens: accounts table, create-user dialog, custom role editor with capability checkboxes from GET /capabilities, identity mappings, and project access. Show the effect of a role change on the next request."

Exit: a 5-turn conversation resolves references; intent accuracy is 95% or higher; an approved SQL pair is retrieved on a similar question; OIDC login works through Keycloak; a profiling run on a changed demo schema produces a drift report; a custom role built in the UI changes access within a minute. **Human checkpoint:** write 10 real follow-up chains from how your analysts talk, and configure the identity mappings for your directory.

Day 7: Streaming API, dashboards, scheduled questions, sharing and the audit browser

Goal: the full backend contract of section 8 (FR-14, FR-20, FR-23, FR-27, FR-28, FR-29).

Track A

- [ ] D7-A1 Final ask endpoint: the SSE contract including `compilation` events, `preview_as` for accounts with the capability, `rerun-sql`, feedback
- [ ] D7-A2 Evaluation endpoints and the first full evaluation of every set on the compiler path
- [ ] D7-A3 Triage failures by stage with the `eval-analyst`

Track B

- [ ] D7-B1 Saved questions, dashboards and tiles (pinned answers with logical SQL and chart spec), layout API; tile results computed under the viewer's principal with a result cache keyed by `policy_context`
- [ ] D7-B2 Scheduled questions: hourly, daily, weekly and cron schedules, run history (last 20), pause and resume, failure state, runs as the owner that fail closed when the owner is disabled
- [ ] D7-B3 Sharing: `shares` checks in every service, revoke, audit events
- [ ] D7-B4 `GET /me`, `shared-with-me`, notification records for failed runs and new drift reports, account unlock and reset-password, the allowed-hosts setting

Track C

- [ ] D7-C1 Audit browser: filter bar, table with paging, detail drawer, CSV export
- [ ] D7-C2 Drift report screen with "create draft from drift"
- [ ] D7-C3 Single sign-on and directory login options on the login page
- [ ] D7-C4 Evaluation screen and the Shared with me screen

Primary prompts

- A: "Implement POST /threads/{id}/ask per the SSE contract in section 8: map LangGraph node updates to events, flush after each event, include the compiled SQL only for accounts with rerun_sql, and emit a terminal error event with a typed failure if

any node raises. Test event order with a fake graph and once with the real graph and a fake LLM."

- B: "Implement dashboards, scheduled questions and sharing per sections 7 and 8. A viewer's tile result is always computed under the viewer's principal; a cached result is reused only when policy_context matches. Write the two-viewer test first."
- C: "Build the Audit browser against GET /audit-events with the filters in section 9, a detail drawer showing the event JSON, and CSV export. Virtualize the table."

Exit (backend contract gate): every endpoint in section 8 is implemented and in the OpenAPI document; the generated 403 tests pass; the shared-dashboard test with two viewers of different policies passes; a scheduled run executes once per slot with two workers running; a job whose owner is disabled pauses itself. **Human checkpoint:** try 5 hostile requests by hand: another project's id, a shared tile viewed under a different policy, a huge `max_rows`, a revoked share, and a malformed stream.

Day 8: Dashboards and schedules in the UI, accuracy and the security review

Goal: finish the collaboration screens, raise accuracy with one lever per run, and complete the second-reviewer security pass (FR-27, FR-28, FR-29, NFR-01, NFR-10).

Track A

- [ ] D8-A1 Pull the accuracy levers in order (section 11), one per evaluation run, and record each delta
- [ ] D8-A2 Rerun the bake-off with the final prompts and confirm the chosen models still meet the targets; try one model size up and one down
- [ ] D8-A3 Sampling or schema linking only if your own evaluation shows a clear gain

Track B

- [ ] D8-B1 Second-reviewer security pass across the compiler, validator, row policies, authentication, sharing and scheduler; fix the findings
- [ ] D8-B2 Performance: retrieval cache, compiled-output cache keyed by `policy_context`, connection pooling, rate limits
- [ ] D8-B3 In-app notices for failed scheduled runs and new drift reports

Track C

- [ ] D8-C1 Dashboards screen: tile grid, pin from chat, refresh, "computed for you" labels
- [ ] D8-C2 Scheduled questions screen: create from chat, schedule picker, run history, pause, run now
- [ ] D8-C3 Sharing dialog for threads, dashboards and questions with user and role pickers

Primary prompts

- A: "Read reports/eval-latest.md. Cluster failures by stage (retrieval miss, generation error, compiler gap, validation gap, bad reference). For the top cluster propose the smallest change, implement it, rerun, and report the accuracy delta. Do not edit golden references."
- B: "Act as the rls-reviewer and auth-reviewer on the whole diff since Day 1. Try to return rows outside a policy through every path in section 10, and to keep access after a demotion, a disabled owner or a revoked share. List findings by severity with a failing test for each."
- C: "Build the Dashboards and Scheduled questions screens from section 9. A tile must show the viewer's own result, label it, and never show another viewer's cached rows."

Exit: a DBA builds a dashboard, schedules a question and shares both; a user and a custom-role account open them read-only and each see only their own rows; the security review has no open blocking findings; accuracy is at or above the baseline with at least one lever applied. **Human checkpoint:** read every finding from the security review and decide the fix order yourself.

Day 9: Integration, end to end and the accuracy gate

Goal: everything works together for every role and login method, and the accuracy target is met. The afternoon is held as buffer for anything that slipped.

- [ ] D9-1 Playwright flows per role: DBA, user, custom role, LDAP user and OIDC user, covering login, asking, dashboards, schedules, sharing and the audit browser
- [ ] D9-2 Full `make eval`: reach the accuracy target and confirm the chosen models still meet the targets
- [ ] D9-3 Offline test: the whole stack with outbound traffic blocked, including the worker, OpenLDAP and Keycloak
- [ ] D9-4 Failure drills: model server down, database down, worker restarted mid-job, clock skew, expired tokens
- [ ] D9-5 Tracing and metrics views: GPU time and tokens per question, scheduler health
- [ ] D9-6 The fix list from reviews and drills

Primary prompt: "Run every suite (check, eval, red-team, row-level security, Playwright, offline). For each failure, name the stage, find the smallest fix, apply it, and rerun only that suite before the full run."

Exit (accuracy gate): execution accuracy is at or above the target set on Day 3; all Playwright flows are green; the red-team and row-level security suites are at 100%; the offline test and the failure drills pass.

Day 10: Packaging, documentation and buffer

Goal: a clean clone runs end to end and the project is handed over.

- [ ] D10-1 Finalize compose: pinned image versions, `.env.example`, `make up`, the identity-demo profile with OpenLDAP and Keycloak, GPU settings documented
- [ ] D10-2 README with a quick start and architecture diagram, a runbook (deploy, re-index, rotate keys, add roles, configure LDAP and OIDC, schedule jobs, reset a DBA) and a 10-minute demo script
- [ ] D10-3 Final evaluation report committed; known-issues list in `docs/`
- [ ] D10-4 Buffer for slipped tickets and bug fixes; tag `v0.1.0`
- [ ] D10-5 Update this document to match what was built

Primary prompt: "Starting from a fresh clone in an empty directory, follow only the README and get to a first answered question, first as a DBA and then as a user. Fix every README or compose problem you hit, then update docs/PROJECT.md to match."

Exit: the definition of done in section 15 is met; a new engineer reaches a first answered question from the README in 15 minutes once the models are pulled.

14. CLAUDE.md template and prompt library

Copy the template into the repo root on Day 1 and edit the details; the prompts below assume it exists. Each track's worktree also gets a short addendum in its own directory (compiler rules next to the compiler, worker rules in `worker/`, design rules in `ui/`).

CLAUDE.md

```
# Text-to-SQL Platform

Semantic-layer text-to-SQL service for PostgreSQL. FastAPI + LangGraph,
Qdrant, local models via Ollama.
Full spec: docs/PROJECT.md (this document). Decisions log: docs/DECISIONS.md.

## Commands
- Install: `uv sync` (UI: `npm --prefix ui install`)
- Run API: `uv run uvicorn app.api.main:app --reload`   Worker: `uv run
python -m app.worker.main`
- Fast tests: `make test-fast`    Full gate: `make check` (ruff, mypy,
import-linter, pytest)
- Eval: `make eval-fast` (cached) / `make eval` (full, real LLM)   Security
suites: `make security`
- Stack: `docker compose up -d` (qdrant, postgres, ollama, demo-db, worker)
- Identity demo: `docker compose --profile identity-demo up -d` (openldap,
keycloak)
```

```
## Architecture rules (hexagonal) - never break
- app/domain: pure Python models, roles and capabilities, policies, errors.
No I/O, no framework imports.
- app/ports: Protocol interfaces only (LLMPort, EmbedderPort,
VectorStorePort, WarehousePort,
  MetadataRepoPort, IdentityPort, CachePort).
- app/application: services, the semantic compiler and the LangGraph agent.
Depends on domain and ports only.
  Nodes receive dependencies by injection; no global clients.
- app/adapters: the only place that imports ollama, qdrant_client, psycopg,
sqlalchemy, ldap3, authlib.
- app/api: routers translate HTTP to services; no business logic.
- import-linter contracts enforce the above; do not weaken them.

## Safety rules - never break
- The only SQL that reaches the warehouse is compiler output that passed the
validator. No bypass flags.
- Row policies are applied by the compiler, never by prompts and never by
filtering rows afterwards.
- Every route declares a capability from domain/auth.py; role and project
checks live in
  dependencies, never inline.
- Every Qdrant and metadata call is filtered by project inside the adapter.
- Passwords, tokens and secrets never appear in logs.

## Conventions
- Python 3.11+, full type hints, Pydantic v2 at every boundary; TypeScript
strict in ui/.
- LLM outputs are structured (Pydantic) through Ollama's format field; never
parse free text with regex.
- Errors are typed (domain/errors.py); never return plain strings for
failures.
- Runtime is local-only: no cloud APIs, no model downloads, no telemetry.
- Prompts live in app/application/agent/prompts/ as versioned templates, not
inline strings.
- Tests first: write a failing test from the acceptance criterion, then
implement.
- Work only in your track's worktree. Contract changes (ports, OpenAPI,
capability names) go through a
  small pull request labelled contract.
- Small conventional commits; one ticket per branch.

## Definition of done
- The acceptance criterion in docs/PROJECT.md section 3 has a passing test.
- `make check` is green; coverage for domain and application stays at 85% or
```

```
higher.
- New behavior has a line in docs/DECISIONS.md when it changes a design
choice.

## Do not touch without asking
- tests/redteam/**, tests/rls/**, tests/eval/golden/** (human-approved)
- .env*, any credentials, docker volumes
- scripts/db/grants.sql and the default capability sets of the system roles

## PostgreSQL notes
- Row limit: LIMIT n. Identifiers are lower case unless quoted. Dates:
date_trunc, intervals, to_char.
- Dry run: EXPLAIN without ANALYZE. Sessions are read-only with
statement_timeout set.
- Policy values are bound parameters (%(name)s), never formatted into the SQL
string.
```

Slash commands to create in `.claude/commands/` (each is a Markdown file whose body is the prompt; `$ARGUMENTS` receives your text)

Command	Prompt it contains
`/ticket`	"Read CLAUDE.md and the docs/PROJECT.md section for ticket $ARGUMENTS. Summarize the acceptance criteria. Enter plan mode: propose files, interfaces, tests and risks. Do not write code until I approve."
`/add-node`	"Add LangGraph node $ARGUMENTS: define its input and output state fields, write failing unit tests with fake ports, implement it, register it in graph.py, and update the topology test."
`/add-adapter`	"Implement an adapter for port $ARGUMENTS: write a contract test any implementation must pass, implement it, add a fake for unit tests, and wire it in the dependency container."
`/add-endpoint`	"Add endpoint $ARGUMENTS contract-first: update the OpenAPI document, declare its capability, write the success, 401, 403 and cross-project tests, implement it, and regenerate the client."

Command	Prompt it contains
`/add-screen`	"Add screen $ARGUMENTS from section 9: build it against the mock API, hide it from accounts without its capability, add a Playwright test for a permitted and a denied account."
`/review-diff`	"Run the architect-reviewer, sql-safety-reviewer, rls-reviewer, auth-reviewer and ui-reviewer subagents that apply to the current git diff; list blocking issues first, then suggestions."
`/eval`	"Run make eval-fast, then use the eval-analyst subagent to cluster failures by stage and propose the smallest fix for the top cluster."
`/day-wrapup`	"Run make check. Summarize what changed today in this track, open issues and the exact first prompt for tomorrow. Append decisions to docs/DECISIONS.md."

Starter prompts for the hardest tickets

- **Compiler (Day 4).** "Implement application/semantic/compiler.py per section 6. Inputs: logical SQL, MDL, principal, policies. Output: CompiledQuery with SQL, bound parameters, policy_context and typed errors. Make the compile test set pass; report any case you could not make pass instead of weakening it."
- **Row-level security (Days 4 and 5).** "Implement row policies per section 6. A missing attribute must produce no rows. Then write the row-level security suite: for each of 3 test users, every query shape in section 10 must return only permitted rows. When a shape leaks, fix the compiler, not the test."
- **LDAP and OIDC (Days 5 and 6).** "Implement the identity adapters per section 8 behind IdentityPort, with contract tests every adapter passes. Use the OpenLDAP and Keycloak containers for integration tests. Never store a directory password."
- **Scheduler (Day 6).** "Implement the worker per sections 7 and 10: APScheduler with jobs stored in the metadata database, a lease row per job slot so two workers never run the same slot, runs as the owner at run time, and fail closed and pause when the owner is disabled. Test with two workers."
- **Dashboards and sharing (Day 7).** "Implement dashboards, scheduled questions and shares per sections 7 and 8. A viewer's result is always computed under the viewer's own principal; write the two-viewer test with different policies first."

- **SSE (Day 7).** "Implement POST /threads/{id}/ask streaming events exactly per the SSE contract in section 8. Add a test that parses the stream and asserts event order, one that asserts a terminal error event on LLM failure, and one that asserts compiled SQL appears only for accounts with rerun_sql."
- **Evaluation harness (Days 3 and 5).** "Build the evaluation runner: for each golden case run the agent, execute reference and generated SQL through the compiler, compare result sets order-insensitively, and write JSON and Markdown reports with a diff against the previous run."

Prompting habits that save time

- Name the file, the function and the section number; ask for one deliverable per message.
- Ask Claude Code to state assumptions before coding and to list anything it could not verify.
- Tell it how it will be judged ("done means these 4 tests pass and `make check` is green").
- When output is wrong, ask what part of `CLAUDE.md` or the spec was ambiguous, then fix that text.

15. Risks, assumptions and definition of done

The plan has one slack half-day (the afternoon of Day 9) and a full buffer day (Day 10). The largest risk is the amount of work for ten days, so the mitigations are about keeping three parallel tracks reviewable. Nothing is removed from the plan to recover time; a slipped ticket moves forward on its track.

Risks

Risk	Likelihood	Impact	Mitigation	Trigger and response
The full scope does not fit in ten days	High	High	Three parallel tracks, narrow definitions of done, small tickets, twice-daily merge gates, a second reviewer, the Day 9 afternoon and Day 10 as buffer	A track more than half a day behind at gate 2: the other tracks review and unblock it first; overflow uses the buffer

Risk	Likelihood	Impact	Mitigation	Trigger and response
Review becomes the bottleneck	High	High	Reviewer subagents, tests as the specification, small diffs, two gates a day	Review queue growing: stop starting tickets on the other tracks until it clears
A query shape leaks rows outside a policy	Medium	High	Compiler-injected filtered CTEs with bound parameters, a suite of 60 or more cases, the `rls-reviewer`, fail closed on a missing attribute	Any leak blocks merging; the shape becomes a permanent test case
Hallucinated tables or columns reach execution	Medium	High	The compiler accepts only deployed logical names, the validator, a dry run, typed correction	Validity under 60% on Day 5: tighten retrieval and add few-shot pairs
Thin metadata lowers accuracy	High	High	LLM-drafted descriptions with a review flag; the coverage gate blocks deploy for golden-set models	More than 20% of columns flagged: spend half of Day 2 on review
Local model accuracy ceiling below the target	High	High	Semantic layer and short curated context, SQL pairs and instructions, the correction loop, the bake-off, a larger model on stronger hardware	Baseline below the provisional goal on Day 3: do metadata and SQL-pair work before touching prompts, then try the next model size

Risk	Likelihood	Impact	Mitigation	Trigger and response
Structured output fails on a small model	Medium	High	Ollama JSON-schema format, schema repeated in the prompt, one repair retry, the bake-off rejects unreliable models	More than 2 in 100 structured calls fail: switch model or simplify the schema
Hardware below the reference tier	Medium	High	Confirm the tier before Day 1; tier A is for development only	Latency more than twice the budget: step down a model size or relax the targets explicitly
A route ships without a permission check	Medium	High	Capability table, route-walk test, generated 403 tests	Any route found without a capability blocks the release
LDAP or OIDC differs from the demo containers	Medium	Medium	Contract tests every identity adapter passes; mappings kept in the database; demo containers from Day 5	A real directory behaves differently: adjust filters and mappings, not the code structure
Scheduler double runs or missed runs	Medium	Medium	A lease per slot, run history, the two-worker and restart tests	A duplicate run found: block the release
Canvas and dashboard UI take longer than planned	Medium	Medium	React Flow and a grid library, mock API from Day 1, `ui-reviewer`	Behind at the Day 8 gate: Track A and B reviewers take the first block of Day 9 for Track C
Golden set too easy or wrong	Medium	High	Human-approved references; difficulty tags; thumbs-down cases added	Accuracy above 95% on Day 3: add harder cases before trusting it

Risk	Likelihood	Impact	Mitigation	Trigger and response
A dependency breaks the local-only rule	Low	High	Egress test on an internal network; telemetry off; models pulled once	Any container reaches the internet: fail the build

Assumptions to confirm

- A machine at or above reference tier B (section 4) runs the models, with Docker installed.
- One engineer reviews three Claude Code sessions, ideally with a second reviewer for Track B.
- A representative PostgreSQL schema (10 to 40 tables) and a read-only database account are available by Day 1.
- You can supply 40 real questions or approve drafts for the golden set by Day 3, and define 3 test users with attributes by Day 4.
- A directory (LDAP) and an OIDC provider on your network are available for acceptance, or the demo containers are accepted for it.
- An embedding model is chosen on Day 3 and fixed; changing it later means a full re-index.

Dependencies

- A Docker host for Qdrant, the metadata and demo PostgreSQL databases, Ollama (with GPU drivers if a GPU is used), the worker, and OpenLDAP and Keycloak for the identity demo.
- A network path from the development environment to your real PostgreSQL server, or a copy of its schema.
- Self-hosted Langfuse in compose, or structured logs only.
- A local reverse proxy with a certificate (for example Caddy or nginx) if the app is used from other machines.

Open decisions (settle by the day shown; the default applies if undecided)

Decision	Options	Needed by	Default
Reference hardware	Tier A, B, C or D (section 4)	Day 1	Tier B

Decision	Options	Needed by	Default
Embedding model	`bge-m3`, `nomic-embed-text` or `mxbai-embed-large` (all local)	Day 3	The best scorer in the bake-off, with the dimension fixed afterward
SQL generation model	Qwen2.5-Coder 14B or 32B, or whichever wins the bake-off	Day 3	Best accuracy per GB of memory in the bake-off
Golden-set source	Real analyst questions, or Claude Code drafts you approve	Day 3	Approved drafts plus 10 real questions
Identity providers wired first	LDAP, OIDC or both	Day 5	Both, tested against the demo containers
Role mappings	Which directory groups and claims map to `dba`, `user` and custom roles	Day 6	The DBA group maps to `dba`; everyone else maps to `user`
DBA access to other users' threads	Allowed for DBAs in their projects, or private to each user	Day 7	Allowed for DBAs in their projects
Read-only model browser for users	Show model and column descriptions to users, or hide them	Day 8	Hidden; users only ask
Tracing provider	Self-hosted Langfuse, or structured logs only	Day 9	Langfuse in compose
Sample datasets	E-commerce and Human Resources, or other schemas of your choice	Day 1	E-commerce and Human Resources

Roles and approvals (assign names before Day 1; one person may hold several roles)

Role	Responsible for	Approves
Engineer (owner)	The daily loop across the three tracks, every diff, the compiler, validator and prompts	Merges and releases
Claude Code (three sessions)	Implementation, tests, drafts of golden cases and docs	Nothing; every change is reviewed

Role	Responsible for	Approves
Domain expert	Descriptions of the key tables, golden-set references, follow-up chains	Golden set and instructions
Database owner	The read-only role, grants and network path	Grants and connection details
Identity administrator	LDAP and OIDC settings, group names and attributes	Identity mappings
Second reviewer	The Day 8 security pass: compiler, row policies, authentication, sharing, scheduler	Release readiness

Definition of done for the 10 days

- A DBA can sign in (password, LDAP or single sign-on), connect a PostgreSQL database, introspect it, model it on the canvas, define metrics, deploy, add knowledge and row policies, schedule profiling, build a dashboard, and schedule and share a question; a user can sign in, ask questions and open what is shared with them.
- Questions stream steps, SQL, rows, a short answer and optionally a chart; follow-ups work across at least 5 turns.
- No SQL other than a compiled, validated single `SELECT` ever executes; the red-team set passes 100%.
- Zero rows outside policy across the row-level security suite, including dashboards, scheduled runs and shares.
- `make eval` reports execution accuracy at or above the Day 3 target and the report is committed.
- Every endpoint declares a capability; the route-walk and generated 403 tests pass.
- `docker compose up` starts the full stack and it answers a question with outbound network blocked; the README, this document and the OpenAPI spec are current.

16. Appendix

Environment variables (all configuration enters through `app/config.py`, a Pydantic settings class; `.env.example` is committed, `.env` is not; real PostgreSQL databases are added as projects in the UI, and their connection strings are stored encrypted)

Variable	Example	Purpose
`OLLAMA_BASE_URL`	`http://ollama:11434`	Local model server address
`LLM_MODEL_SQL`, `LLM_MODEL_FAST`	model tags pulled into Ollama, chosen in the bake-off	Generation and correction; intent, rewrite and summary
`EMBEDDING_MODEL`, `EMBEDDING_DIM`	local embedding model tag, fixed dimension	Embeddings; changing them forces a re-index
`QDRANT_URL`, `QDRANT_API_KEY`	`http://qdrant:6333`	Vector store
`METADATA_DB_URL`	`postgresql+psycopg://app:...@postgres/app`	Projects, models, accounts, policies, history
`WAREHOUSE_DSN_DEMO`	`postgresql://readonly:...@demo-db:5432/demo`	Demo warehouse for seeds and tests
`CONNECTION_ENCRYPTION_KEY`	(secret, Fernet)	Encrypts stored connection strings
`JWT_SECRET` or `JWT_PUBLIC_KEY`	(secret)	Signs and verifies access tokens (a secret for HS256, or a key pair for RS256)
`ACCESS_TOKEN_TTL_MIN`, `REFRESH_TOKEN_TTL_HOURS`	`15`, `8`	Token lifetimes (provisional defaults)
`LOGIN_MAX_FAILURES`, `LOGIN_LOCK_MINUTES`	`5`, `15`	Lockout after repeated failed logins (provisional defaults)
`COOKIE_SECURE`	`true`	Marks the refresh cookie Secure; set false only for plain-HTTP localhost development
`BOOTSTRAP_DBA_USERNAME`, `BOOTSTRAP_DBA_PASSWORD`	`admin`, (secret)	Creates the first DBA on first start; remove both afterwards

Variable	Example	Purpose
`LDAP_URL`, `LDAP_BIND_DN`, `LDAP_BIND_PASSWORD`, `LDAP_USER_BASE`, `LDAP_USER_FILTER`, `LDAP_GROUP_BASE`	`ldaps://ldap.example.internal`, …	Directory login; leave `LDAP_URL` empty to disable
`OIDC_ISSUER`, `OIDC_CLIENT_ID`, `OIDC_CLIENT_SECRET`, `OIDC_REDIRECT_URL`	`https://sso.example.internal/realms/main`, …	Single sign-on; leave `OIDC_ISSUER` empty to disable
`QUERY_TIMEOUT_S`, `MAX_ROWS`	`30`, `1000`	Execution limits
`MAX_CORRECTIONS`	`3`	Retry cap in the graph
`RESULT_CACHE_TTL_S`, `RUN_HISTORY_KEEP`	`300`, `20`	Cached dashboard results; scheduled runs kept per question
`WORKER_LEASE_S`	`60`	Job lease that prevents double runs
`OLLAMA_KEEP_ALIVE`	`24h`	Keeps models resident so requests skip the load time
`OFFLINE_MODE`	`true`	Blocks model downloads and enables the egress test
`TRACING_PROVIDER`, `LANGFUSE_*`	`langfuse`	Observability
`LOG_LEVEL`	`INFO`	Logging
DB_HOST_ALLOWLIST	10.0.0.0/8,192.168.0.0/16,demo-db	Initial allowed hosts and ranges for the connection form, edited in the app afterwards; link-local and metadata addresses are always refused

docker-compose.yml (services)

```
services:
  api:
    build: .
    env_file: .env
    ports: ["8000:8000"]
    depends_on: [qdrant, postgres, ollama]
  worker:
    build: .
    command: python -m app.worker.main
    env_file: .env
    depends_on: [qdrant, postgres, ollama]
  ui:
    build: ./ui
    environment: { API_BASE_URL: http://api:8000 }
    ports: ["3000:3000"]
  qdrant:
    image: qdrant/qdrant:latest
    ports: ["6333:6333"]
    volumes: ["qdrant_data:/qdrant/storage"]
  postgres:
    image: postgres:16
    environment: { POSTGRES_USER: app, POSTGRES_PASSWORD: app, POSTGRES_DB:
app }
    volumes: ["pg_data:/var/lib/postgresql/data"]
  ollama:
    image: ollama/ollama
    ports: ["11434:11434"]
    volumes: ["ollama_models:/root/.ollama"]
    # GPU: add the NVIDIA device reservation, or run Ollama on the host
  demo-db:
    image: postgres:16
    environment: { POSTGRES_USER: demo, POSTGRES_PASSWORD: demo, POSTGRES_DB:
demo }
    volumes: ["./scripts/demo/seed.sql:/docker-entrypoint-initdb.d/seed.sql"]
  openldap:
    image: osixia/openldap
    profiles: ["identity-demo"]
  keycloak:
    image: quay.io/keycloak/keycloak
    command: start-dev
    profiles: ["identity-demo"]
    ports: ["8080:8080"]
volumes: { qdrant_data: {}, pg_data: {}, ollama_models: {} }
```

Pin image versions before sharing the repository; `latest` is acceptable only for the first local run. The OpenLDAP and Keycloak services exist for testing the identity adapters and are not part of a production deployment.

Makefile targets

Target	Does
`make check`	`ruff check`, `ruff format --check`, `mypy`, `lint-imports`, `pytest` with the coverage gate
`make test-fast`	Unit tests only, no network
`make security`	Route-walk, generated 403, red-team and row-level security suites
`make eval-fast`	10 golden cases using the recorded LLM cache, plus the compile test set
`make eval`	Every evaluation set against the real LLM; writes `reports/eval-<date>.md`
`make seed`	Loads the demo schema and data
`make index`	Re-indexes the active MDL of a project
`make up` / `make down`	Start and stop the compose stack
`make models`	Pulls the configured models into Ollama (an online step, run once)
`make egress-test`	Starts the stack on an internal network and fails if any container reaches the internet
`make user-create`	Creates an account with a role and a temporary password from the command line
`make user-reset-dba`	Resets a DBA password when every DBA is locked out

Glossary

Term	Meaning
MDL	Modeling Definition Language: the JSON semantic model of models, columns, relationships, metrics and canvas layout

Term	Meaning
Logical model	A business-level table defined in the MDL; the only thing the LLM and users can refer to
Semantic compiler	The component that turns SQL over logical models into the PostgreSQL SQL that actually runs, applying row policies
Closed vocabulary	The rule that generated SQL may only use names present in the MDL
SQL pair	A verified question and SQL example used as a few-shot and retrieval target; proposed by users, approved by a DBA
Instruction	A business rule injected into prompts, for example the fiscal-year start
Dry run	Asking the database to plan a query (`EXPLAIN`) without returning data
Execution accuracy	Share of questions where generated and reference SQL return the same result set
Typed failure	A structured error with a code, message and retryable flag, used for correction and for the UI
Port and adapter	An interface owned by the core, and the concrete integration that implements it
Bake-off	A side-by-side evaluation of candidate local models on the same cases, prompts and context
Ollama	The local model server that runs the open-weight language and embedding models
Reference hardware	The machine tier (section 4) on which the accuracy and latency targets are measured
DBA	Role that manages connections, the semantic model, knowledge, policies, schedules, dashboards, evaluations and accounts
User	Role that can only log in, ask questions in assigned projects, rate answers and open what is shared
Custom role	A role a DBA builds from the capability list

Term	Meaning
Capability	A named permission such as `ask_question` or `deploy_mdl`, mapped to roles in the database
Row policy	A predicate on a model, built from user attributes, that the compiler adds to every query on that model
Policy context	A hash of the row policies and attribute values applied to a query; results are cached and shared only within the same context
Drift	The difference between the live PostgreSQL schema and the deployed semantic model
Worker	The process that runs scheduled profiling and scheduled questions, with a lease so a slot runs once
Access token	Short-lived signed token sent as a bearer header on each API call
Refresh token	Longer-lived random value in an httpOnly cookie, rotated on each use, that obtains new access tokens
Setup wizard	The three-step flow (connect, select tables, define relationships) a DBA uses to add a PostgreSQL database